

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр»
за спеціальністю 121 «Інженерія програмного забезпечення»
(освітня програма «Інженерія програмного забезпечення»)

на тему:

«Вебплатформа для автомобільних майданчиків»

Виконав студент групи ВТ-22-1
БЕКІРОВ Аліхан Елмуродович

Керівник роботи:
ВЛАСЕНКО Олег Васильович

Рецензент:
НЕРОДА Сергій Іванович

Житомир – 2026

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

«ЗАТВЕРДЖУЮ»

Зав. кафедри інженерії
програмного забезпечення

Тетяна ВАКАЛЮК

«27» лютого 2026 р.

ЗАВДАННЯ
на кваліфікаційну роботу

Здобувач вищої освіти: **БЕКІРОВ Аліхан Елмуродович**

Керівник роботи: **ВЛАСЕНКО Олег Васильович**

Тема роботи: **«Вебплатформа для автомобільних майданчиків»**,
затверджена Наказом закладу вищої освіти від **«27» лютого 2026 р., №92/с**

Вихідні дані для роботи: процеси обліку автомобілів, обробки заявок, супроводу
доставки та взаємодії користувачів з автомобільним майданчиком

Консультанти з бакалаврської кваліфікаційної роботи із зазначенням розділів,
що їх стосуються:

Розділ	Консультант	Завдання видав	Завдання прийняв
1	Власенко О.В.	01.03.2026р.	01.03.2026р
2	Власенко О.В.	01.03.2026р	01.03.2026р
3	Власенко О.В.	01.03.2026р	01.03.2026р

РЕФЕРАТ

Випускна кваліфікаційна робота бакалавра складається з вебдодатка для автомобільних майданчиків та пояснювальної записки. Пояснювальна записка містить 74 сторінок, 35 рисунків, 13 таблиць, 2 додатки та 30 використаних джерел.

Метою роботи є розробка вебплатформи для автомобільних майданчиків, яка автоматизує облік автомобілів, пошук і перегляд пропозицій, оформлення заявок, супровід доставки, адміністрування каталогу, ведення блогу та інформування користувачів.

У роботі використано методи системного аналізу, порівняльного аналізу аналогів, об'єктно-орієнтованого проєктування, моделювання даних, проєктування REST API, компонентного підходу до побудови клієнтської частини та функціонального тестування.

У результаті виконання роботи розроблено клієнт-серверний вебзастосунок на основі Symfony, Vue, Doctrine ORM і MySQL. Реалізовано каталог автомобілів, розмежування ролей, JWT-автентифікацію, обране, порівняння, заявки, трекінг доставки, сповіщення, збережені пошуки, блог і адміністративну панель.

КЛЮЧОВІ СЛОВА: АВТОМОБІЛЬ, ВЕБПЛАТФОРМА, ДОСТАВКА, ЗАЯВКА, КАТАЛОГ

					ДУ «Житомирська політехніка».26.121.03.000 - ПЗ			
Змн.	Арк.	№ докум.	Підпис	Дата	Випускна кваліфікаційна робота бакалавра			
Розроб.	Бекіров А.Е.							
Керівник.	Власенко О.В.							
Н. контр.								
Затвердив					ФІКТ, Гр. ВТ-22-1			
					Літ.	Арк.	Аркушів	
						2	74	

ABSTRACT

The bachelor qualification work consists of a web application for car lots and an explanatory note. The explanatory note contains 74 pages, 35 figures, 13 tables, 2 appendices and 30 references.

The purpose of the work is to develop a web platform for car lots that automates vehicle accounting, searching and viewing offers, order processing, delivery support, catalogue administration, blog management and user notification.

The work uses methods of system analysis, comparative analysis of analogues, object-oriented design, data modelling, REST API design, component-based client-side development and functional testing.

As a result, a client-server web application based on Symfony, Vue, Doctrine ORM and MySQL was developed. The system implements a vehicle catalogue, role-based access control, JWT authentication, favourites, comparison, orders, delivery tracking, notifications, saved searches, a blog and an administrative panel.

KEYWORDS: CAR, CATALOGUE, DELIVERY, ORDER, WEB PLATFORM.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		4

ЗМІСТ

РЕФЕРАТ	3
ABSTRACT	4
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	6
ВСТУП.....	7
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ВИБІР ЗАСОБІВ РЕАЛІЗАЦІЇ	9
1.1. Постановка задачі.....	9
1.2. Аналіз наявних аналогів програмного продукту	11
1.3. Вибір архітектури вебплатформи	15
1.4. Обґрунтування вибору інструментальних засобів та вимоги до апаратного забезпечення	19
Висновки до першого розділу.....	22
РОЗДІЛ 2. ПРОЄКТУВАННЯ ВЕБПЛАТФОРМИ ДЛЯ АВТОМОБІЛЬНИХ МАЙДАНЧИКІВ	23
2.1. Специфікація вимог до програмного продукту	23
2.2. Проєктування інформаційної моделі та бази даних	27
2.3. Проєктування архітектури та алгоритмів роботи платформи	30
2.4. Реалізація застосунку вебплатформи	38
2.5. Об'єктно-орієнтована та компонентна будова програмного застосунку	42
Висновки до другого розділу	50
РОЗДІЛ 3. ІНТЕРФЕЙС ТА ПОРЯДОК РОБОТИ З ВЕБПЛАТФОРМОЮ ДЛЯ АВТОМОБІЛЬНИХ МАЙДАНЧИКІВ	51
3.1. Порядок встановлення та налаштування параметрів системи	51
3.2. Структура інтерфейсу та порядок роботи з вебплатформою	54
3.3. Тестування роботи програмного комплексу	65
Висновки до третього розділу.....	69
ВИСНОВКИ.....	70
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	72
ДОДАТКИ.....	75

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		5

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API – прикладний програмний інтерфейс, через який клієнтська частина взаємодіє із сервером.

БД – база даних.

CRUD – базові операції створення, перегляду, оновлення та видалення даних.

DTO – об’єкт передавання даних між шарами програмної системи.

HTTP – протокол передавання даних у вебсередовищі.

JWT – токен, що використовується для безпечної автентифікації користувача.

MVC – архітектурний патерн поділу системи на модель, представлення та контролер.

ORM – технологія відображення об’єктної моделі застосунку на реляційну базу даних.

REST – архітектурний підхід до побудови вебсервісів на основі ресурсів і стандартних HTTP-методів.

SPA – односторінковий вебзастосунок.

UI – користувацький інтерфейс.

UML – уніфікована мова моделювання програмних систем.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		6

ВСТУП

Сучасний автомобільний ринок активно переходить у цифрове середовище, оскільки покупці все частіше шукають транспортні засоби через онлайн-каталоги, маркетплейси та спеціалізовані вебсервіси. Автомобільні майданчики, дилерські центри та компанії, що займаються продажем автомобілів з наявності або під замовлення, потребують зручних інструментів для публікації пропозицій, ведення обліку транспортних засобів, контролю статусів, приймання заявок, супроводу доставки та комунікації з клієнтами [4].

Актуальність теми зумовлена тим, що традиційне ведення обліку автомобілів у таблицях, месенджерах або окремих неінтегрованих сервісах ускладнює роботу автомобільного майданчика. Працівникам доводиться окремо оновлювати статуси авто, перевіряти актуальність цін, зберігати історію змін, відповідати на запити клієнтів і контролювати процес доставки автомобілів під замовлення. Для користувача така модель також є незручною, оскільки інформація про автомобіль, його статус, технічні характеристики, ціну, фото, доступність і можливість оформлення заявки часто подається неповно або розміщується на різних ресурсах [28].

Метою випускної кваліфікаційної роботи бакалавра є розробка вебплатформи для автомобільних майданчиків, яка забезпечує публікацію та адміністрування автомобілів, облік їх статусів, оформлення заявок, супровід доставки, перегляд історії змін, роботу з обраним, порівнянням, збереженими пошуками, сповіщеннями та блогом.

Для досягнення поставленої мети необхідно виконати аналіз предметної області, дослідити наявні аналоги, сформулювати вимоги до майбутньої системи, обґрунтувати вибір архітектури та інструментальних засобів, спроектувати базу даних, реалізувати серверний API, створити клієнтський інтерфейс і перевірити працездатність основних сценаріїв використання.

Об'єктом дослідження є процес організації діяльності автомобільного майданчика у вебсередовищі. Цей процес охоплює публікацію інформації про автомобілі, ведення їх обліку, зміну статусів, приймання заявок від користувачів,

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		7

відстеження доставки автомобілів під замовлення та взаємодію між адміністрацією платформи й клієнтами.

Предметом дослідження є програмні засоби, архітектурні рішення, моделі даних і механізми реалізації вебплатформи для автомобільних майданчиків. У межах предмета дослідження розглядаються способи побудови клієнт-серверної архітектури, організації REST API, автентифікації користувачів, адміністрування даних, фільтрації автомобілів, збереження історії змін, обробки заявок і відображення інформації у зручному інтерфейсі.

У роботі використано методи системного аналізу, порівняльного аналізу аналогів, об'єктно-орієнтованого проектування, моделювання баз даних, вебпрограмування, проектування REST API, рольового розмежування доступу та функціонального тестування. Практичне значення отриманих результатів полягає у створенні програмного продукту, який може використовуватися як основа для автоматизації роботи автомобільного майданчика або автодилера.

Пояснювальна записка складається зі вступу, трьох розділів, висновків, списку використаних джерел і додатків. У першому розділі виконано аналіз предметної області та аналогів, у другому розділі описано проектування системи, а у третьому розділі наведено інтерфейс, порядок роботи й тестування вебплатформи. Бібліографічний опис використаних джерел оформлено відповідно до вимог ДСТУ 8302:2015 [30].

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		8

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ВИБІР ЗАСОБІВ РЕАЛІЗАЦІЇ

1.1. Постановка задачі

Автомобільний майданчик є організацією або онлайн-представництвом, яке займається розміщенням, продажем, демонстрацією та супроводом автомобілів. У межах такого майданчика можуть одночасно перебувати автомобілі з різними статусами: у наявності, продано, під замовлення, у дорозі або зарезервовано. Для кожного автомобіля потрібно зберігати технічні характеристики, комерційні параметри, фотографії, дату надходження, ціну, опис, інформацію про поточний статус.

Проблема полягає в тому, що значна частина автомобільних майданчиків використовує розрізнені інструменти для ведення обліку та взаємодії з клієнтами. Інформація про автомобілі може зберігатися в таблицях, оголошеннях на сторонніх сайтах, месенджерах або локальних документах. Через це виникають помилки під час оновлення статусів, дублювання інформації, втрата історії змін, складність контролю заявок і відсутність єдиного зручного інтерфейсу для користувача.

Особливо важливою ця проблема стає для майданчиків, які працюють не лише з автомобілями в наявності, а й з авто під замовлення. У такому випадку клієнт очікує прозорого відображення процесу доставки, розуміння етапів переміщення автомобіля, орієнтовних термінів і можливості отримувати оновлення без постійного звернення до менеджера.

Розроблювана вебплатформа повинна вирішувати ці проблеми шляхом створення єдиної інформаційної системи, у якій адміністрація майданчика зможе керувати автомобілями, заявками, доставкою, користувачами, блогом і сповіщеннями. Користувачі платформи повинні мати можливість переглядати каталог автомобілів, користуватися фільтрами, відкривати детальну сторінку авто, додавати транспортні засоби до обраного, порівнювати кілька пропозицій, створювати заявки та переглядати інформацію про власні замовлення.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		9

У системі необхідно передбачити розмежування доступу між звичайним користувачем та адміністратором. Звичайний користувач працює з публічною частиною вебплатформи, переглядає каталог, читає блог, формує заявки та користується особистими функціями. Адміністратор отримує доступ до панелі керування, де може створювати, редагувати та видаляти автомобілі, змінювати їх статуси, переглядати заявки, оновлювати етапи доставки, працювати із записами блогу та сповіщеннями.

Результатом виконання роботи має стати вебзастосунок, який складається з серверної та клієнтської частин. Серверна частина відповідає за обробку запитів, роботу з базою даних, автентифікацію, авторизацію, бізнес-логіку та формування відповідей API. Клієнтська частина відповідає за відображення сторінок, інтерактивну взаємодію з користувачем, роботу з формами, фільтрами, картками автомобілів, особистим кабінетом і адміністративним інтерфейсом.

До складу задачі також входить забезпечення базових нефункціональних властивостей. Система повинна бути достатньо надійною для повсякденної роботи майданчика, захищати адміністративні функції від неавторизованого доступу, коректно обробляти помилкові дані та залишатися зручною для подальшого розширення. Під час формування вимог було враховано підхід до інженерії вимог, за яким функціональність системи повинна бути пов'язана з реальними потребами користувачів і бізнес-процесами предметної області [28].

У межах роботи задача обмежується розробкою вебплатформи, яка працює як клієнт-серверний застосунок. Передбачається, що користувач взаємодіє із системою через браузер, а серверна частина зберігає дані, перевіряє права доступу, обробляє запити та повертає відповіді у форматі JSON. Така постановка задачі відповідає вимогам до сучасних вебзастосунків і дозволяє в майбутньому підключити інші клієнтські інтерфейси.

З погляду адміністратора система повинна зменшити кількість ручних операцій. У традиційній моделі працівник майданчика може одночасно вести таблицю автомобілів, окремо приймати заявки в месенджері, вручну повідомляти клієнта про доставку та додатково публікувати інформаційні

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		10

матеріали. Такий підхід створює ризик помилок, тому розроблювана платформа повинна централізувати ці процеси в одному застосунку.

З погляду користувача система повинна бути простою та передбачуваною. Покупець не повинен знати внутрішню структуру автомобільного майданчика, але має отримувати достатньо інформації для ухвалення рішення. До такої інформації належать фотографії, технічні характеристики, ціна, статус, місце розташування, дата надходження, можливість порівняння та створення заявки. Усі ці дії мають виконуватися через зрозумілий інтерфейс без потреби звертатися до адміністратора для отримання базових даних.

Важливою особливістю предметної області є залежність між станом автомобіля, заявками користувачів і діями адміністратора. Наприклад, автомобіль може бути доступним у каталозі, потім перейти у стан резервування, після цього стати проданим або перейти в етап доставки. Якщо такі зміни не фіксувати окремо, адміністрація втрачає можливість відстежити історію роботи з автомобілем. Саме тому в постановці задачі передбачено збереження історії зміни ціни, статусу, відповідальної особи та етапів доставки.

Під час постановки задачі враховано, що автомобільний майданчик поєднує публічні та внутрішні бізнес-процеси. Публічна частина спрямована на покупця, який переглядає каталог і порівнює пропозиції. Внутрішня частина призначена для адміністратора, який підтримує актуальність даних, змінює статуси автомобілів, працює із заявками та контролює доставку. Тому система не може обмежуватися лише сторінкою з оголошеннями, а повинна мати повноцінну інформаційну модель і набір службових модулів.

1.2. Аналіз наявних аналогів програмного продукту

Перед розробкою вебплатформи для автомобільних майданчиків доцільно проаналізувати наявні онлайн-сервіси, які використовуються для пошуку, продажу та перегляду автомобілів. Такі ресурси дають змогу визначити типові функції, які очікує користувач від сучасної автомобільної платформи, а також виявити недоліки, які можна врахувати під час створення власного програмного продукту.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		11

Одним з найпоширеніших українських сервісів для пошуку автомобілів є AUTO.RIA. Платформа орієнтована на розміщення та перегляд оголошень про продаж транспортних засобів, а також містить інструменти перевірки автомобіля за VIN-кодом або державним номером [1]. Для користувача перевагою такого сервісу є велика кількість оголошень, можливість фільтрації, перегляду фото, характеристик, ціни та контактної інформації продавця.

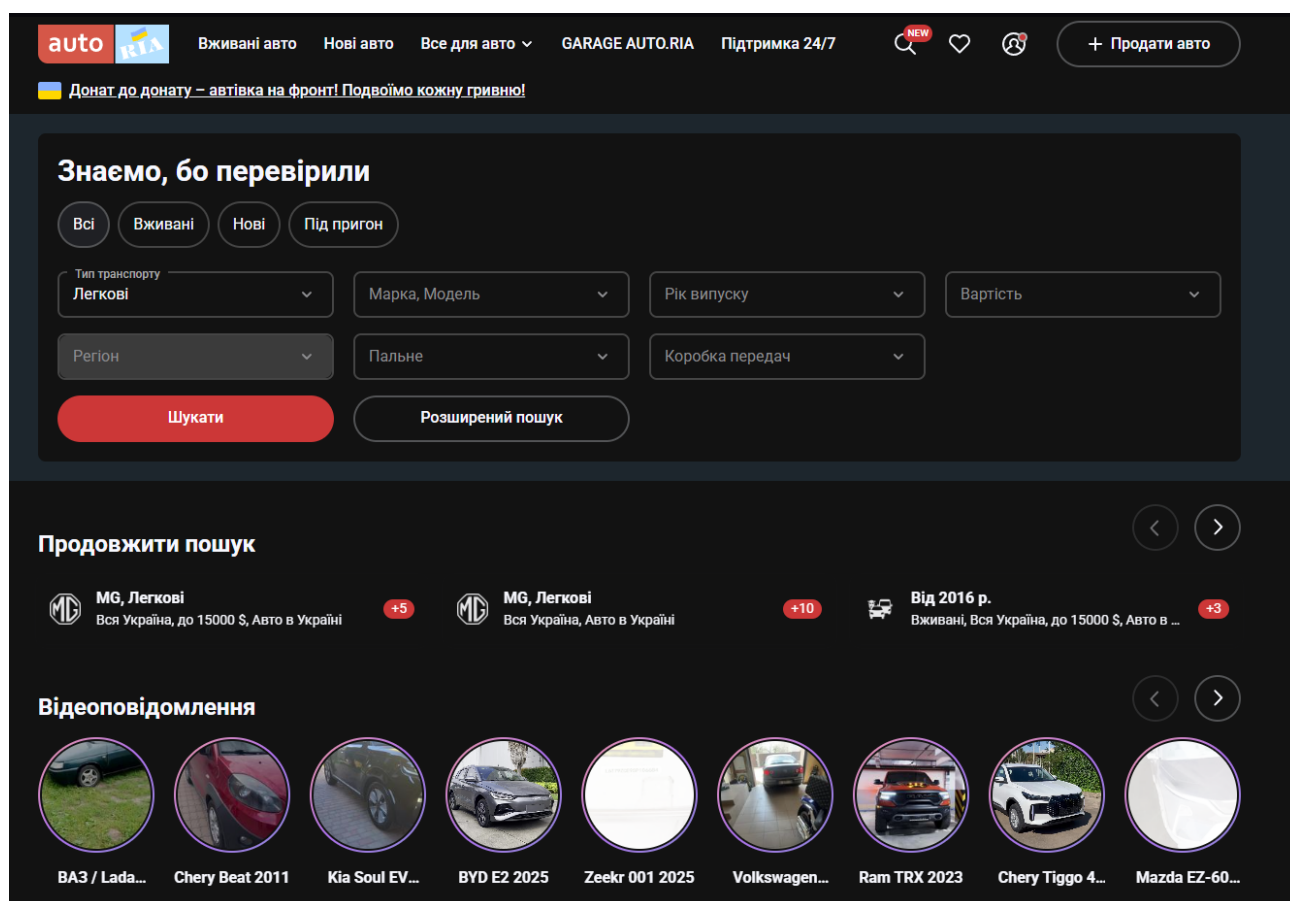


Рисунок 1.1 – Інтерфейс сервісу AUTO.RIA

Незважаючи на широку функціональність, AUTO.RIA більше орієнтований на публікацію оголошень, ніж на внутрішній облік роботи окремого автомобільного майданчика. У межах дипломного проєкту важливо не лише показати автомобілі користувачу, а й реалізувати адміністративну частину, історію змін, статуси, заявки та доставку.

Іншим популярним сервісом є OLX Авто, який використовується для розміщення приватних і комерційних оголошень про продаж автомобілів. Його сильна сторона полягає у простоті публікації оголошень і широкій аудиторії

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		12

користувачів. Користувач може швидко знайти автомобіль за базовими параметрами, переглянути опис і зв'язатися з продавцем [2].

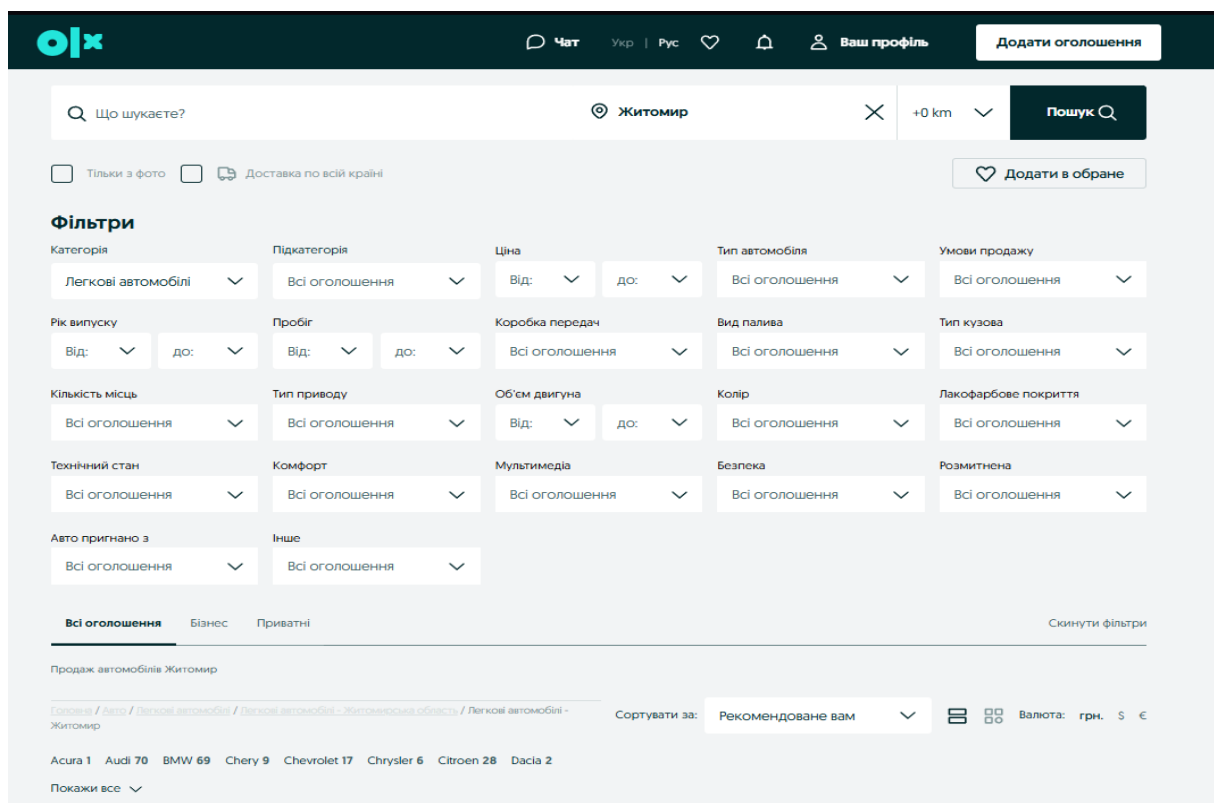


Рисунок 1.2 – Інтерфейс автомобільного розділу OLX Авто

Недоліком такого підходу є те, що сервіс не є спеціалізованою системою управління автомобільним майданчиком. Він не вирішує завдання повного обліку автомобілів, контролю доставки, збереження історії змін і комплексної аналітики для адміністратора. Для невеликого продавця цього може бути достатньо, але для майданчика з великою кількістю автомобілів потрібна більш цілісна система.

Copart є міжнародною платформою онлайн-аукціонів автомобілів. Вона надає доступ до лотів, інформації про транспортні засоби, статусів продажу та процесу участі в торгах. Для розроблюваної системи важливою є ідея поетапного супроводу автомобіля, оскільки автомобілі під замовлення проходять кілька станів від вибору й купівлі до доставки клієнту. Водночас Copart не є локальною системою автомобільного майданчика з власним блогом, кабінетом користувача, обраним, порівнянням і повною адміністративною панеллю [3]. Інтерфейс сервісу Copart наведено на рисунку 1.3.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		13

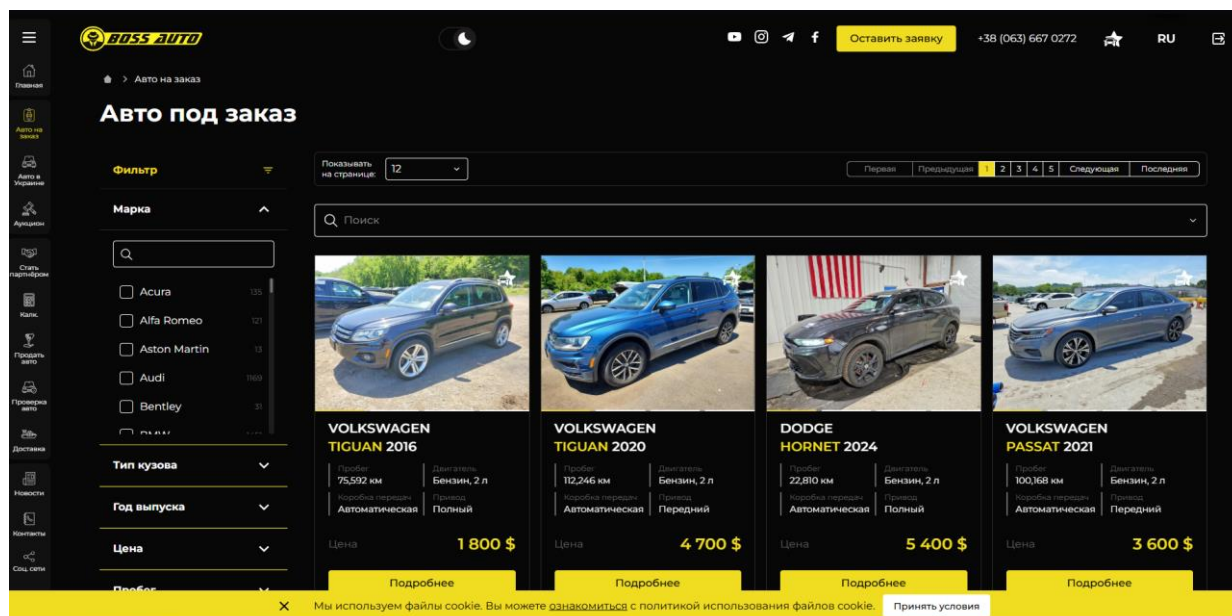


Рисунок 1.3 – Сторінка сервісу Sorpart [3]

Проведений аналіз показує, що кожний аналог закриває лише окрему частину задач. Автомобільні каталоги зручні для пошуку, аукціонні платформи демонструють важливість статусів і етапів, CRM-системи допомагають працювати із заявками, однак для невеликого автомобільного майданчика доцільно об'єднати ці можливості в одному додатку. Порівняння функціональних можливостей наведено в табл. 1.1.

Таблиця 1.1

Порівняння аналогів і власної вебплатформи

Функціональна можливість	AUTO.RIA	OLX Авто	Sorpart	Власна вебплатформа
Каталог автомобілів	+	+	+/-	+
Розширена фільтрація за параметрами	+	+/-	+	+
Внутрішній облік автомобілів	-	-	-	+
Історія зміни ціни, статусу та відповідальної особи	-	-	-	+
Заявки клієнтів і статуси обробки	+/-	+/-	-	+

Трекінг доставки автомобіля	-	-	+/-	+
Обране та порівняння автомобілів	+	+/-	-	+
Блог та інформаційні матеріали	+/-	-	-	+
Адміністративна панель під конкретний майданчик	-	-	-	+

У таблиці 1.1 узагальнено результати порівняння аналогів. На основі цього порівняння визначено, що власна розробка повинна мати не тільки стандартні можливості каталогу, а й функції прозорого супроводу автомобіля та замовлення, які зазвичай розподілені між кількома окремими системами.

1.3. Вибір архітектури вебплатформи

Для реалізації вебплатформи для автомобільних майданчиків доцільно використати клієнт-серверну архітектуру. Такий підхід передбачає поділ системи на дві основні частини: серверну, яка відповідає за бізнес-логіку, роботу з базою даних і обробку запитів, та клієнтську, яка забезпечує взаємодію користувача з інтерфейсом. Клієнтська частина надсилає запити до серверного API, отримує дані у форматі JSON і відображає їх у вигляді сторінок, форм, списків, карток автомобілів та адміністративних таблиць [27].

Серверна частина вебплатформи реалізується на основі фреймворку Symfony. Цей фреймворк підтримує побудову структурованих вебзастосунків, роботу з маршрутами, контролерами, сервісами, безпекою та іншими компонентами, необхідними для створення повноцінного API [5]. У межах платформи серверна частина обробляє запити на отримання списку автомобілів, створення та редагування транспортних засобів, завантаження зображень, оформлення заявок, оновлення етапів доставки, роботу з користувачами, сповіщеннями, блогом та іншими модулями.

Клієнтська частина створюється з використанням Vue. Обраний підхід дає змогу будувати інтерфейс у вигляді окремих компонентів, які можна повторно використовувати на різних сторінках [8; 9]. Наприклад, картка автомобіля, форма фільтрації, таблиця заявок, модальне вікно підтвердження, блок сповіщень або елемент навігації можуть бути реалізовані як окремі компоненти.

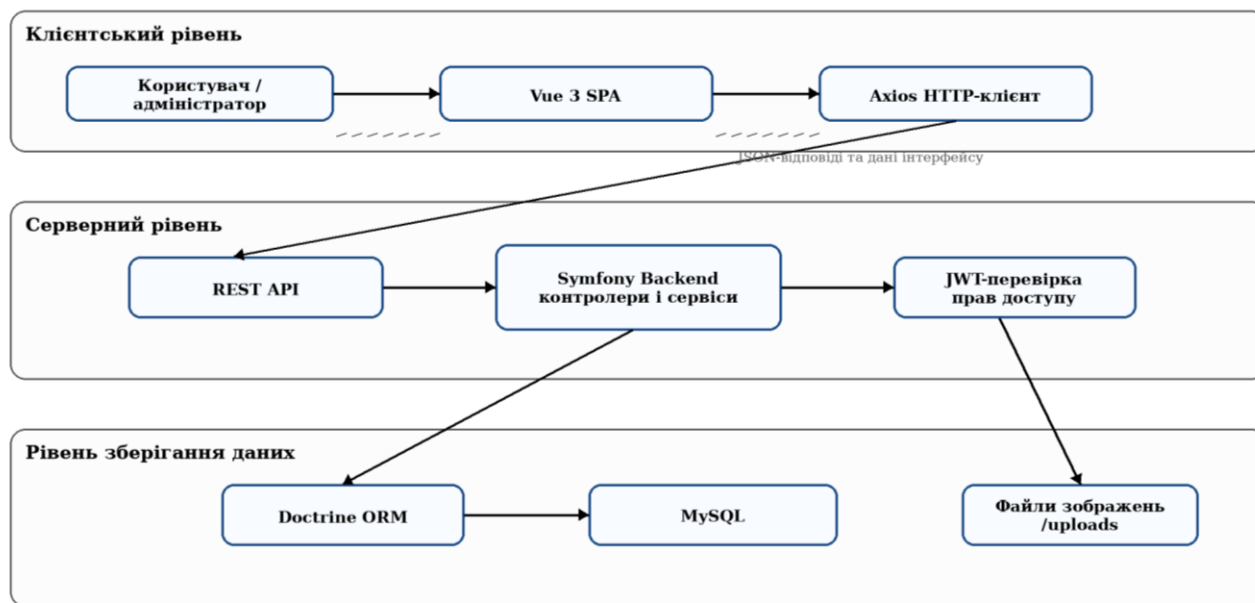


Рисунок 1.4 – Загальна архітектура вебплатформи для автомобільних майданчиків

Для взаємодії між клієнтом і сервером використовується REST API. Такий підхід є зручним для вебплатформи, оскільки кожна сутність системи може бути представлена як окремий ресурс. Автомобілі, категорії, заявки, користувачі, повідомлення, обране, порівняння та записи блогу можуть мати власні API-маршрути [15; 16].

Для зберігання даних використовується реляційна база даних MySQL. Такий вибір є обґрунтованим, оскільки дані вебплатформи мають чітку структуру та пов'язані між собою. Наприклад, автомобіль може мати категорію, набір зображень, історію змін, заявки, записи доставки та взаємодію з користувачами [12].

Обрана архітектура також передбачає використання механізмів автентифікації та авторизації. Для доступу до особистих і адміністративних

функцій користувач повинен пройти автентифікацію. Після входу до системи клієнтська частина отримує токен і використовує його під час подальших запитів до захищених API-маршрутів [14; 17].

Загалом вибрана архітектура дозволяє розглядати платформу як сукупність взаємопов'язаних, але логічно відокремлених підсистем. Її можна описати через представлення користувача, логічне представлення модулів, представлення процесів обробки заявок і фізичне представлення розгортання. Такий спосіб аналізу узгоджується з підходом архітектурних представлень програмних систем [27].

У проєкті передбачено окремий напрям роботи із завантаженими файлами. Фотографії автомобілів не зберігаються безпосередньо в таблицях бази даних, а розміщуються в каталозі файлів, тоді як у базі зберігаються назви файлів, ознака головного зображення та зв'язок із автомобілем. Такий підхід зменшує навантаження на базу даних і спрощує відображення галереї на клієнтській частині.

Клієнтська частина побудована як SPA, тому після початкового завантаження сторінки більшість дій виконується без повного перезавантаження браузера. Це особливо корисно для каталогу автомобілів, де користувач часто змінює фільтри, відкриває сторінки, повертається до результатів пошуку або переходить до порівняння. Компонентна модель Vue дає змогу повторно використовувати картки автомобілів, форми, кнопки, таблиці та блоки сповіщень [8].

У серверній частині архітектура наближена до типового підходу Model-Controller-Service-Repository. Контролер приймає HTTP-запит, сервіс виконує прикладну логіку, репозиторій відповідає за пошук або вибірку даних, а сутність описує структуру об'єкта предметної області. Цей підхід дозволяє не перевантажувати контролери бізнес-логікою та краще відокремлювати різні відповідальності в коді.

Для зменшення складності реалізації застосовано модульний підхід. Кожна функціональна зона має власні контролери, сутності та, за потреби, сервіси.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		17

Каталог автомобілів відокремлений від заявок, заявки від доставки, доставка від сповіщень, а блог від основного каталогу. Такий принцип відповідає ідеї слабкої зв'язаності компонентів і полегшує підтримку програмного продукту після завершення розробки.

Архітектурна модель також враховує наявність декількох типів користувачів. Гість взаємодіє лише з відкритими сторінками, авторизований користувач отримує доступ до особистих функцій, а адміністратор працює зі службовими модулями. На серверному рівні це реалізується через перевірку токена та ролі користувача. Завдяки цьому одна й та сама платформа може безпечно обслуговувати публічний каталог і адміністративну панель.

У межах архітектури важливо відокремити маршрути читання від маршрутів зміни даних. Запити на перегляд каталогу, детальної сторінки автомобіля, блогу або власних заявок використовують логіку отримання даних. Запити на створення автомобіля, оформлення заявки, зміну статусу або додавання етапу доставки змінюють стан системи та потребують суворішої перевірки ролей. Такий поділ робить поведінку API більш передбачуваною та відповідає принципам REST-архітектури [15].

Для опису архітектури доцільно використовувати багаторівневий підхід. На рівні представлення знаходяться сторінки та компоненти Vue, які формують користувацький і адміністративний інтерфейси. На рівні прикладної логіки розташовані контролери Symfony, сервіси та правила обробки даних. На рівні доступу до даних працюють сутності, репозиторії та Doctrine ORM. Окремо виділяється рівень зберігання, представлений базою даних MySQL і каталогом завантажених зображень.

Обрана архітектура розглядається не лише як технічна схема з'єднання клієнта й сервера, а як спосіб розподілу відповідальності між частинами системи. Клієнтський рівень відповідає за відображення інформації, збирання введених користувачем даних, локальну навігацію та надсилання запитів. Серверний рівень приймає ці запити, виконує перевірку доступу, запускає бізнес-логіку,

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		18

працює з базою даних і повертає результат. Такий розподіл зменшує зв'язність між інтерфейсом і внутрішньою логікою платформи.

1.4. Обґрунтування вибору інструментальних засобів та вимоги до апаратного забезпечення

Під час розробки вебплатформи для автомобільних майданчиків було обрано набір інструментів, які забезпечують зручну реалізацію серверної частини, клієнтського інтерфейсу, бази даних і взаємодії між модулями системи. Основними критеріями вибору були придатність для створення вебзастосунків, підтримка клієнт-серверної архітектури, можливість роботи з базою даних, наявність засобів безпеки, зручність підтримки коду та можливість подальшого масштабування.

Для серверної частини обрано мову програмування PHP та фреймворк Symfony. PHP широко використовується для створення вебзастосунків, а Symfony надає структурований підхід до побудови проєкту. У межах платформи Symfony використовується для організації маршрутів, контролерів, сервісів, обробки HTTP-запитів, налаштування безпеки, роботи з базою даних і формування API-відповідей [5; 13].

Для клієнтської частини обрано Vue. Цей фреймворк дає змогу створювати динамічний інтерфейс, який швидко реагує на дії користувача. У межах вебплатформи Vue використовується для побудови сторінки каталогу, детальної сторінки автомобіля, форм авторизації та реєстрації, особистого кабінету, адміністративної панелі, списків заявок, модулів обраного, порівняння та сповіщень [8; 10].

Для зберігання даних використовується MySQL, а для взаємодії серверної частини з базою даних застосовується Doctrine ORM. Його використання дозволяє описати таблиці бази даних у вигляді сутностей і працювати з ними як з об'єктами, що спрощує реалізацію бізнес-логіки та підтримку коду [6; 12].

Для реалізації автентифікації використовується JWT-підхід. Він дозволяє клієнтській частині отримати токен після успішного входу користувача та

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		19

використовувати його під час звернення до захищених маршрутів. Завдяки цьому сервер може перевіряти права доступу та визначати роль користувача [7; 14].

Таблиця 1.2

Інструментальні засоби розробки

Засіб	Призначення	Обґрунтування вибору
Symfony	Серверна частина та REST API	Забезпечує структуровану організацію контролерів, сервісів, безпеки та роботи з базою даних
Vue	Клієнтська частина	Дозволяє створювати компонентний інтерфейс і зручно працювати зі станом сторінок
MySQL	База даних	Підходить для зберігання структурованих пов'язаних даних автомобільної платформи
Doctrine ORM	Об'єктно-реляційне відображення	Спрощує роботу з сутностями та міграціями бази даних
JWT	Автентифікація	Дає змогу організувати захищений доступ до API та рольове розмежування
Vite	Середовище збірки клієнта	Забезпечує швидкий запуск і збирання Vue-застосунку

Під час експлуатації найбільший вплив на ресурси системи матимуть кількість зображень автомобілів, обсяг каталогу та кількість одночасних запитів. Тому в архітектурі передбачено зберігання зображень у файловій системі, використання пагінації каталогу та фільтрацію на серверному рівні. Таке рішення дозволяє зберігати прийнятну продуктивність навіть після збільшення кількості автомобілів у базі.

Для роботи системи достатньо типового середовища веброботи. На сервері або локальному комп'ютері повинні бути встановлені PHP, Composer, MySQL, Node.js та npm. Мінімальні апаратні вимоги передбачають багатоядерний процесор початкового або середнього рівня, не менше 8 ГБ оперативної пам'яті для комфортної розробки, достатній обсяг дискового простору для бази даних, завантажених зображень і залежностей проекту.

Axios застосовується для надсилання HTTP-запитів із клієнтської частини до API. Через нього Vue-компоненти отримують список автомобілів, надсилають

форми заявок, завантажують дані профілю, позначають сповіщення як прочитані та виконують інші операції. Наявність єдиного HTTP-клієнта дозволяє централізовано додавати токен авторизації та обробляти помилки відповіді сервера [11].

Vite використовується як інструмент збирання клієнтської частини. Його перевагою є швидкий запуск середовища розробки, підтримка сучасних модулів JavaScript і зручне оновлення сторінки під час зміни файлів. Це прискорює розробку інтерфейсу та полегшує тестування сторінок без потреби кожного разу виконувати повне збирання проєкту [10].

Vue обрано для клієнтської частини, оскільки він дозволяє створювати динамічний інтерфейс із компонентною структурою. Для платформи це важливо, адже багато елементів повторюються на різних сторінках: картки автомобілів, форми фільтрації, таблиці, кнопки дій, блоки повідомлень і модальні вікна. Компонентний підхід зменшує дублювання коду і полегшує зміну дизайну окремих частин інтерфейсу.

MySQL використовується як система керування базами даних через реляційний характер інформації. Автомобіль має зв'язки з майданчиком, зображеннями, заявками, історією статусів і ціни, а користувач пов'язаний із заявками, сповіщеннями та обраним. Реляційна модель дозволяє описати ці зв'язки через первинні та зовнішні ключі, що підвищує цілісність даних [12].

Doctrine ORM обрано для зменшення кількості ручних SQL-запитів і роботи з даними через об'єкти. У межах проєкту це особливо зручно, оскільки предметна область містить багато пов'язаних сутностей: автомобілі, майданчики, заявки, користувачі, історія змін, доставка, сповіщення та публікації блогу. Використання ORM спрощує опис зв'язків, створення міграцій і підтримку структури бази даних [6].

PHP використовується як основна мова серверної частини. Його перевагою для цього проєкту є широка підтримка вебсередовища, сумісність із популярними хостингами, велика кількість бібліотек і зрозумілий процес розгортання. Крім того, PHP добре поєднується з Symfony і Doctrine ORM, що

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		21

дозволяє розробляти структуровані вебзастосунки з поділом на сутності, контролери та сервіси [13].

Під час вибору інструментів враховано не тільки їхню популярність, а й відповідність конкретним задачам платформи. Для системи автомобільного майданчика важливо мати стабільний серверний фреймворк, зручний механізм маршрутизації, підтримку авторизації, роботу з реляційною базою даних і можливість будувати API. Саме тому серверна частина реалізована на Symfony, який надає готову інфраструктуру для організації контролерів, сервісів, конфігурації та безпеки [5].

Висновки до першого розділу

У першому розділі було розглянуто предметну область вебплатформи для автомобільних майданчиків і визначено основні проблеми, які виникають під час організації продажу, обліку та супроводу автомобілів у цифровому середовищі. Було встановлено, що автомобільний майданчик потребує не лише публічного каталогу, а й внутрішньої системи керування автомобілями, заявками, статусами, доставкою, історією змін і взаємодією з користувачами.

Під час аналізу аналогів було визначено, що наявні автомобільні онлайн-сервіси переважно орієнтовані на розміщення оголошень і пошук автомобілів. Вони мають корисні функції фільтрації, перегляду детальної інформації, обраного або порівняння, проте не повністю вирішують завдання автоматизації роботи конкретного автомобільного майданчика.

Також у розділі було обґрунтовано вибір клієнт-серверної архітектури, REST API, реляційної бази даних і набору інструментальних засобів для реалізації програмного продукту. Обрані рішення відповідають вимогам до системи та створюють основу для подальшого проєктування й реалізації вебплатформи.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		22

РОЗДІЛ 2. ПРОЄКТУВАННЯ ВЕБПЛАТФОРМИ ДЛЯ АВТОМОБІЛЬНИХ МАЙДАНЧИКІВ

2.1. Специфікація вимог до програмного продукту

Проектування вебплатформи для автомобільних майданчиків розпочинається з уточнення вимог до майбутнього програмного продукту. У межах цієї роботи система розглядається не як звичайна дошка оголошень, а як комплексна вебплатформа, що поєднує публічний каталог автомобілів, особистий кабінет користувача, адміністративну панель, модуль заявок, супровід доставки, історію змін, блог і допоміжні сервіси інформування. Такий підхід відповідає особливостям предметної області, оскільки автомобільний майданчик повинен не тільки показувати пропозиції покупцю, а й підтримувати внутрішній облік автомобілів та контроль бізнес-процесів.

Система повинна забезпечувати взаємодію трьох основних груп користувачів. Першою групою є гостеві відвідувачі, які переходять на платформу для ознайомлення з каталогом, перегляду автомобілів, читання інформаційних матеріалів і пошуку пропозицій за параметрами. Другою групою є зареєстровані користувачі, які мають доступ до персональних можливостей: створення заявок, додавання автомобілів до обраного, порівняння пропозицій, перегляд власних замовлень, збереження пошукових запитів і отримання сповіщень. Третьою групою є адміністратори, які відповідають за наповнення каталогу, керування статусами, обробку заявок, оновлення доставки, роботу з блогом, перегляд аналітичних показників і підтримку актуальності даних.

Для покупця головною вимогою є швидкий і зрозумілий доступ до інформації про автомобіль. Картка автомобіля повинна містити основні комерційні дані, а детальна сторінка має відображати технічні характеристики, фотографії, опис, статус, місце розташування на майданчику, ціну, дату надходження та доступні дії. Користувачеві потрібно бачити, чи автомобіль є в наявності, проданий, перебуває в дорозі або доступний під замовлення. Для автомобілів під замовлення важливим є прозоре відображення етапів доставки,

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		23

оскільки клієнт очікує зрозумілої інформації про стан замовлення та орієнтовні терміни.

Для адміністратора ключовою вимогою є можливість керувати всіма основними даними без прямого втручання в базу даних або код програми. Адміністративна частина повинна дозволяти створювати автомобілі, редагувати їх технічні та комерційні параметри, завантажувати зображення, змінювати статуси, прив'язувати автомобілі до конкретного майданчика, переглядати історію зміни ціни, статусів і власників, а також опрацьовувати заявки клієнтів. Окремо система повинна підтримувати роботу з автомобільними майданчиками, оскільки один програмний продукт може містити декілька локацій із різними контактними даними та адресами.

Вимоги до системи подано не лише як перелік функцій, а як відображення реальних процесів автомобільного майданчика. Кожна функція повинна мати практичне призначення: каталог забезпечує публікацію пропозицій, заявки формують канал взаємодії з клієнтом, доставка робить процес замовлення прозорим, історія змін дозволяє контролювати динаміку автомобіля, а сповіщення зменшують ризик пропуску важливої події. На рисунку 2.1 показано узагальнену діаграму основних сценаріїв використання платформи.

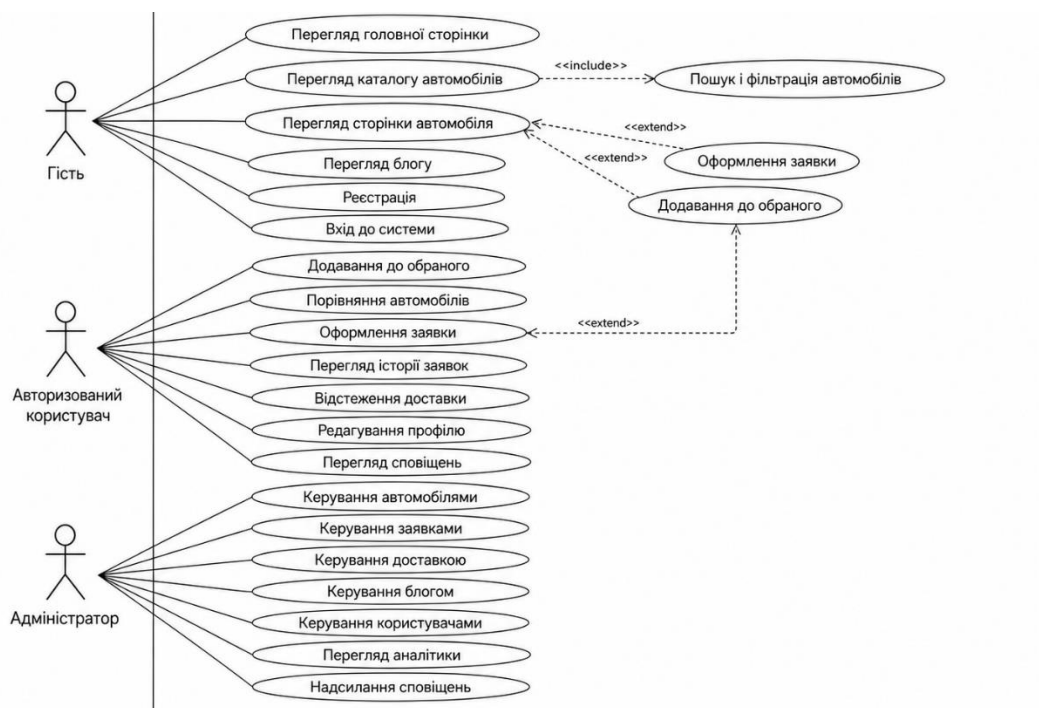


Рисунок 2.1 – Діаграма основних сценаріїв використання вебплатформи

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		24

Узагальнені функціональні вимоги наведено в таблиці 2.1. Таблиця відображає основні модулі платформи та пояснює, для якої групи користувачів вони призначені. Такий формат обрано для того, щоб уникнути надмірного подання вимог у вигляді коротких списків і водночас зберегти чіткість опису.

Таблиця 2.1

Функціональні вимоги до вебплатформи

Модуль	Опис вимоги	Користувач
Каталог автомобілів	Система повинна відображати список автомобілів із фото, ціною, статусом, короткими характеристиками та можливістю переходу до детальної сторінки	Гість, користувач
Пошук і фільтрація	Система повинна дозволяти знаходити автомобілі за маркою, моделлю, роком, ціною, типом пального, коробкою передач, приводом, кузовом, статусом і майданчиком	Гість, користувач
Детальна сторінка авто	Система повинна показувати технічні та комерційні параметри, галерею, опис, статус, локацію, історичні дані та доступні дії користувача	Гість, користувач
Заявки	Користувач повинен мати можливість створити заявку на автомобіль, а адміністратор повинен переглядати її, змінювати статус і залишати коментар	Користувач, адміністратор
Доставка	Адміністратор повинен оновлювати поточний етап доставки, орієнтовну дату та коментар	Користувач, адміністратор
Історія змін	Платформа повинна зберігати історію зміни ціни, статусів, власників та етапів доставки для контролю життєвого циклу автомобіля	Адміністратор
Обране та порівняння	Користувач повинен зберігати цікаві автомобілі та порівнювати декілька пропозицій за основними характеристиками	Користувач
Збережені пошуки	Користувач повинен мати можливість зберігати параметри пошуку та отримувати повідомлення про появу нових відповідних автомобілів	Користувач
Блог	Платформа повинна містити окремий розділ для інформаційних публікацій, категорій, коментарів і реакцій користувачів	Гість, користувач, адміністратор
Аналітика	Адміністратор повинен мати доступ до узагальнених показників щодо автомобілів, заявок і статусів роботи майданчика	Адміністратор

Нефункціональні вимоги визначають якість роботи системи та її придатність до подальшого використання. Для розроблюваної вебплатформи особливо важливими є безпека, продуктивність, зручність, підтримуваність і масштабованість. Безпека забезпечується перевіркою ролей і захистом адміністративних маршрутів, продуктивність досягається оптимізацією запитів і використанням пагінації, а підтримуваність формується завдяки поділу коду на контролери, сутності, репозиторії, сервіси та компоненти клієнтської частини [29].

Таблиця 2.2

Нефункціональні вимоги до вебплатформи

Вимога	Опис	Спосіб забезпечення
Безпека	Службові функції повинні бути недоступними для звичайних користувачів, а запити до API мають проходити перевірку прав	JWT-автентифікація, перевірка ролей, валідація вхідних даних
Продуктивність	Каталог, сторінка автомобіля та адміністративні таблиці повинні відкриватися без значних затримок	Пагінація, фільтрація на сервері, оптимізація зображень і запитів
Зручність використання	Інтерфейс повинен бути зрозумілим для покупця та працівника майданчика без додаткового навчання	Компонентна структура, логічна навігація, повідомлення про результат дій
Надійність	Система повинна коректно обробляти помилкові запити, відсутні дані та спроби несанкціонованого доступу	Перевірка параметрів, обробка помилок, контроль статусів
Підтримуваність	Проект повинен мати структуру, яка дозволяє змінювати окремі модулі без повного переписування системи	Поділ на контролери, сервіси, сутності, репозиторії та Vue-компоненти
Масштабованість	Платформа повинна дозволяти додавання нових модулів, наприклад повідомлень, аналітики або інтеграцій зі сторонніми сервісами	REST API, реляційна модель даних, модульна архітектура

Після визначення функціональних і нефункціональних вимог можна перейти до проектування інформаційної моделі. Вимоги безпосередньо впливають на структуру бази даних, оскільки кожен модуль системи потребує відповідних сутностей, зв'язків і обмежень. Саме тому база даних проектується

з урахуванням не лише поточного каталогу, а й майбутнього розширення функціональності.

2.2. Проектування інформаційної моделі та бази даних

База даних є центральною частиною вебплатформи, оскільки вона зберігає інформацію про автомобілі, користувачів, автомобільні майданчики, заявки, доставку, історію змін, публікації блогу та персональні дії користувачів. Під час її проектування використано реляційну модель, яка дозволяє явно описати зв'язки між основними об'єктами предметної області. Такий підхід є доцільним для платформи автомобільного майданчика, оскільки більшість даних має чітку структуру та логічні залежності.

Основною сутністю системи є Car. Вона описує транспортний засіб і містить технічні, комерційні та службові параметри. До технічних параметрів належать марка, модель, рік випуску, VIN-код, пробіг, тип пального, коробка передач, тип приводу, об'єм двигуна, потужність, тип кузова та колір. До комерційних параметрів належать ціна, валюта, статус, дата надходження, локація та опис. Такий набір даних дозволяє не тільки відобразити автомобіль у каталозі, а й виконувати фільтрацію, сортування та формувати детальну сторінку пропозиції.

Сутність CarLot використовується для опису автомобільного майданчика як окремої локації. Вона містить назву, місто, адресу, телефон, години роботи та опис. Зв'язок між CarLot і Car дозволяє показувати, на якому саме майданчику знаходиться автомобіль, а також у майбутньому розширити систему до багатофіліальної структури. Це важливо для реальних автомобільних компаній, які можуть мати декілька локацій або окремі майданчики для різних типів автомобілів.

Сутність User відповідає за зберігання облікових записів користувачів. Вона містить електронну пошту, пароль, ім'я та роль. Роль визначає набір доступних дій у системі. Звичайний користувач може створювати заявки, додавати автомобілі до обраного, зберігати пошукові параметри та переглядати сповіщення. Адміністратор отримує доступ до службових маршрутів і може

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		27

керувати автомобілями, заявками, доставкою, блогом та іншими адміністративними даними.

Сутність CarOrder пов'язує користувача з конкретним автомобілем. Вона фіксує факт зацікавленості клієнта в автомобілі або замовленні, містить статус заявки, телефон клієнта, коментар користувача, коментар адміністратора та службові дати створення й оновлення. Оскільки заявка є основним каналом взаємодії між покупцем і майданчиком, ця сутність має важливе значення для подальшої роботи адміністратора.

Для супроводу автомобілів під замовлення використовується сутність DeliveryTracking. Вона пов'язана із заявкою та містить поточний етап доставки, орієнтовну дату отримання, примітку, дату створення й дату оновлення. Окремо використовується DeliveryStageHistory, яка зберігає історію проходження етапів. Завдяки цьому система не просто показує останній стан доставки, а й зберігає послідовність змін, що підвищує прозорість процесу для користувача.

Для контролю життєвого циклу автомобіля передбачено сутності CarPriceHistory, CarStatusHistory і CarOwnerHistory. Історія ціни дозволяє зберігати попередню та нову вартість автомобіля, історія статусів фіксує перехід між станами, а історія власників або відповідальних осіб дає змогу бачити зміни, пов'язані з комерційним супроводом авто. Таке рішення робить платформу більш придатною для реального обліку, оскільки важливі зміни не втрачаються після редагування основної картки.

Персоналізовані можливості користувача реалізуються через сутності FavoriteCar, SavedCarSearch і Notification. FavoriteCar зберігає список обраних автомобілів, SavedCarSearch містить параметри збереженого пошуку, а Notification використовується для інформування про створення заявки, зміну статусу або оновлення доставки. Для блогу використовуються сутності Post, Category, Comment і PostLike, що дозволяє винести інформаційні матеріали в окремий розділ і не змішувати їх з каталогом автомобілів.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		28

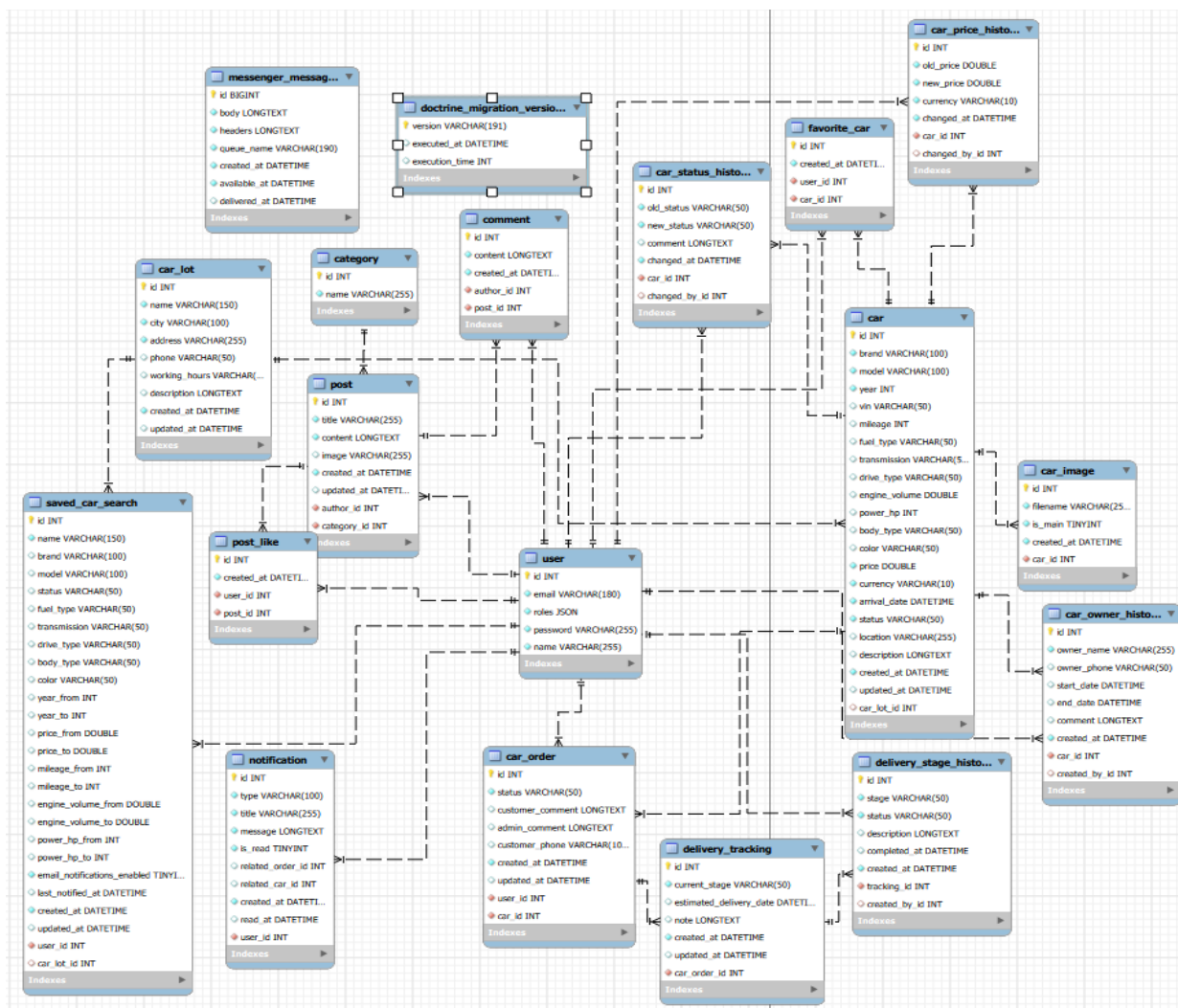


Рисунок 2.2 – Інформаційна модель бази даних вебплатформи

На рисунку 2.2 показано узагальнену інформаційну модель системи. Вона демонструє зв'язки між користувачами, автомобілями, майданчиками, заявками, доставкою, історичними даними, блогом і персональними функціями. Основні сутності бази даних подано в таблиці 2.3.

Таблиця 2.3

Основні сутності бази даних

Сутність	Призначення	Основні поля
User	Зберігання облікових записів і ролей користувачів	id, email, password, roles, name
Car	Зберігання технічних і комерційних даних автомобіля	brand, model, year, vin, mileage, price, status, arrivalDate
CarLot	Опис окремого автомобільного майданчика або філії	name, city, address, phone, workingHours

CarImage	Зберігання зображень автомобіля та позначення головного фото	car, filename, isMain, createdAt
CarOrder	Фіксація заявки користувача на конкретний автомобіль	user, car, status, customerPhone, customerComment, adminComment
DeliveryTracking	Зберігання поточного стану доставки за заявкою	carOrder, currentStage, estimatedDeliveryDate, note
DeliveryStageHistory	Зберігання історії проходження етапів доставки	tracking, stage, status, description, completedAt
CarPriceHistory	Фіксація зміни вартості автомобіля	car, oldPrice, newPrice, currency, changedAt
CarStatusHistory	Фіксація зміни статусу автомобіля	car, oldStatus, newStatus, comment, changedAt
CarOwnerHistory	Зберігання історії власників або відповідальних осіб	car, ownerName, ownerPhone, startDate, endDate
FavoriteCar	Збереження автомобілів в обраному користувача	user, car, createdAt
SavedCarSearch	Збереження параметрів пошуку автомобілів	user, brand, model, status, priceFrom, priceTo
Notification	Інформування користувача про важливі події	user, type, title, message, isRead, createdAt
Post	Зберігання матеріалів блогу	title, content, image, author, category, createdAt
PostLike	Зберігання лайків до публікацій блогу	user, post, createdAt
Category	Зберігання категорій блогу	name
Comment	Зберігання коментарів до публікацій блогу	content, author, post, createdAt

Запропонована структура бази даних забезпечує збереження основних даних і дозволяє реалізувати зв'язки між модулями без дублювання інформації. Наприклад, заявка не дублює всі характеристики автомобіля, а зберігає посилання на відповідний запис Car. Аналогічно етапи доставки пов'язані із заявкою, а не з користувачем напряму, що дозволяє коректно відстежувати доставку саме в межах конкретного замовлення.

2.3. Проектування архітектури та алгоритмів роботи платформи

Архітектура розробленої вебплатформи побудована як клієнт-серверна система з чітким розподілом відповідальності між інтерфейсом користувача, серверною бізнес-логікою та рівнем зберігання даних. Клієнтська частина на Vue

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		30

З виконує роль SPA-застосунку, який відображає сторінки каталогу, особистого кабінету, блогу й адміністративної панелі. Серверна частина на Symfony надає REST API, перевіряє права доступу, виконує валідацію даних, звертається до репозиторіїв Doctrine та повертає клієнту структуровані JSON-відповіді. Під час опису архітектури враховано принципи REST, шарового поділу та подання системи через архітектурні представлення [27].

У проєкті використано багаторівневу архітектуру. Рівень представлення реалізований у Vue-компонентах і відповідає за форми, таблиці, картки автомобілів, фільтри, модальні вікна та повідомлення. Рівень API складається з контролерів Symfony, які приймають HTTP-запити та визначають сценарій обробки. Рівень бізнес-логіки зосереджений у сервісах і методах контролерів, де виконуються перевірки статусів, створення заявок, формування сповіщень і робота зі збереженими пошуками. Рівень доступу до даних забезпечується репозиторіями Doctrine ORM, а фізичне збереження інформації виконується в MySQL. Такий підхід відповідає принципам розділення відповідальності, повторного використання компонентів і підтримуваності коду [26].

Важливою особливістю архітектури є асинхронна взаємодія між SPA та API. Після входу користувач отримує JWT-токен, який клієнтська частина передає в заголовках захищених запитів. Це дозволяє серверу визначити роль користувача та обмежити доступ до адміністративних маршрутів. Такий підхід відокремлює інтерфейс від серверної логіки, спрощує підтримку модулів і дає змогу в майбутньому підключити мобільний застосунок або інший клієнт до того самого API. Захищена взаємодія виконується з урахуванням рекомендацій щодо токенів і контролю доступу [14; 17].

Об'єктно-орієнтована модель серверної частини побудована навколо сутностей User, Car, CarLot, CarOrder, DeliveryTracking, DeliveryStageHistory, Notification, Post і Comment. Сутність Car є центральною для каталогу автомобілів, оскільки з нею пов'язані зображення, заявки, історія статусів, історія ціни, власники та обране. Сутність User використовується для авторизації, персональних функцій, заявок, коментарів і сповіщень. Така

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		31

структура забезпечує логічну цілісність предметної області та дає змогу відстежувати всі ключові зміни в системі.

Алгоритми роботи вебплатформи визначають послідовність дій клієнтської та серверної частин під час виконання ключових сценаріїв. Для розробленої системи найбільш важливими є пошук автомобілів у каталозі, оформлення заявки на автомобіль, супровід доставки, зміна статусу та ціни автомобіля, а також створення сповіщень. Кожен алгоритм охоплює введення даних у Vue-інтерфейсі, обробку запиту REST API на Symfony, перевірку коректності вхідних значень, виконання бізнес-логіки та повернення структурованої відповіді клієнтській частині.

Алгоритм пошуку та фільтрації автомобілів починається з вибору параметрів у каталозі. Користувач може задати пошуковий рядок, марку, модель, статус, тип пального, трансмісію, тип кузова, діапазон років, ціну, пробіг, об'єм двигуна, потужність, колір, тип приводу, майданчик і варіант сортування. Після цього клієнтська частина формує GET-запит до маршруту /api/cars. Контролер CarController передає параметри до репозиторію, який формує вибірку з урахуванням фільтрів, пагінації та сортування. У відповідь користувач отримує масив автомобілів і службову інформацію про сторінки, а Vue-компонент оновлює каталог без повного перезавантаження сторінки.

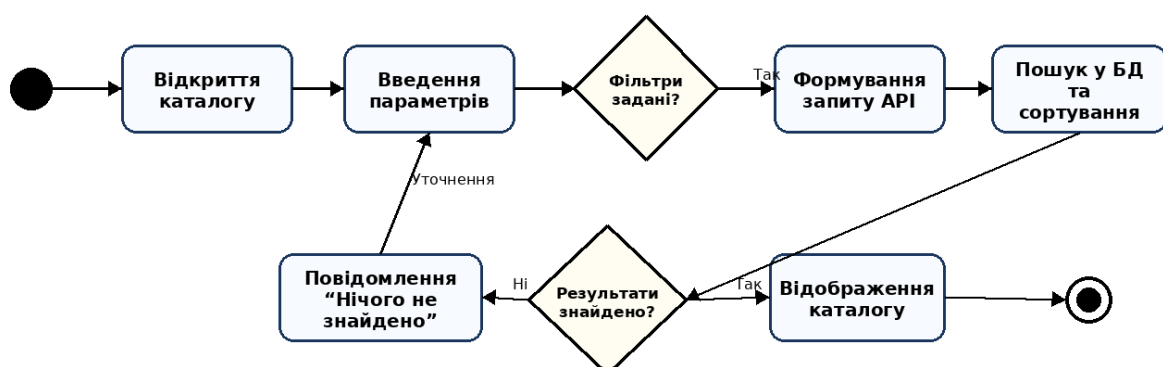


Рисунок 2.3 – Алгоритм пошуку та фільтрації автомобілів

Процес оформлення заявки починається на детальній сторінці автомобіля. Після натискання кнопки оформлення система перевіряє, чи авторизований користувач. Якщо ні, виконується перехід до входу або реєстрації. Далі користувач вводить номер телефону та, за потреби, коментар. Маршрут POST /api/cars/{id}/orders перевіряє наявність автомобіля, його доступність для замовлення, відсутність дублю заявки від цього користувача, коректність номера телефону у форматі +380XXXXXXXXX і довжину коментаря. Після успішної перевірки створюється запис CarOrder зі стартовим статусом, а користувач отримує повідомлення про успішне створення заявки.

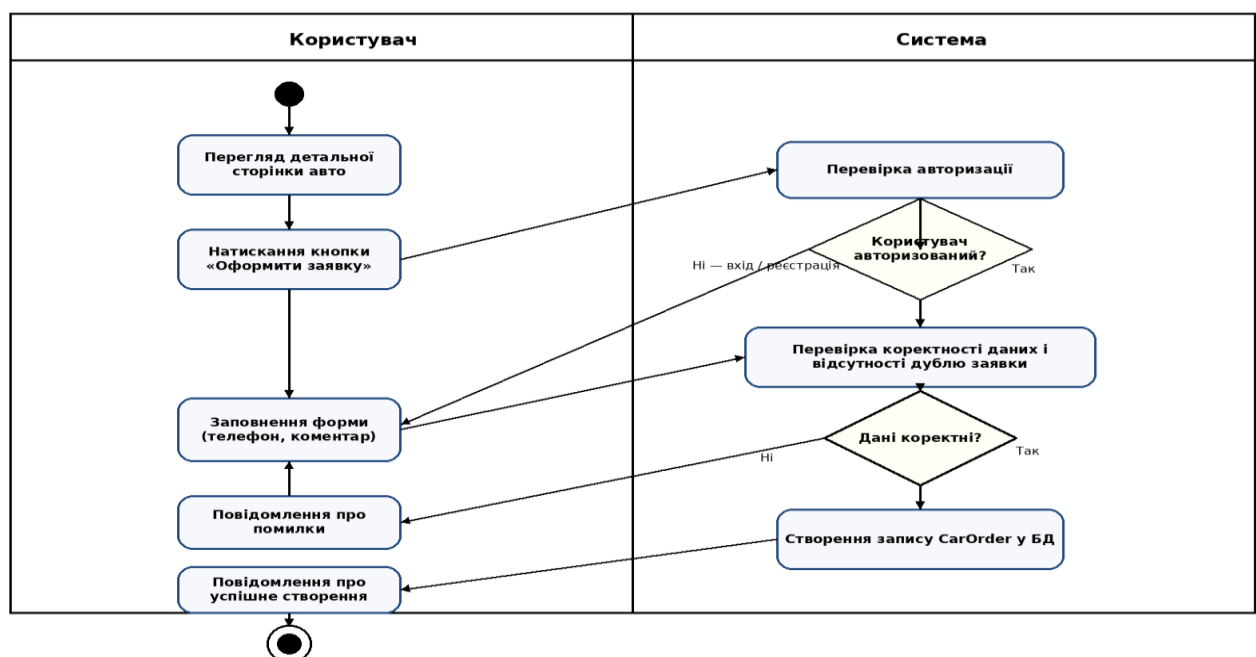


Рисунок 2.4 – Діаграма активності процесу оформлення заявки на автомобіль

Алгоритм супроводу доставки використовується для автомобілів, що оформлюються під замовлення. Адміністратор створює або оновлює поточний трекінг маршрутом POST /api/admin/orders/{id}/tracking, указуючи етап доставки, орієнтовну дату та примітку. Для збереження повної хронології переходів між етапами використовується маршрут /api/admin/orders/{id}/tracking/stages, який створює записи DeliveryStageHistory. Користувач переглядає актуальний стан доставки через GET /api/orders/{id}/tracking. Під час оновлення етапів система синхронізує статус

автомобіля: для проміжних етапів установлюється стан «в дорозі», а після завершення — фінальний стан, передбачений логікою платформи.

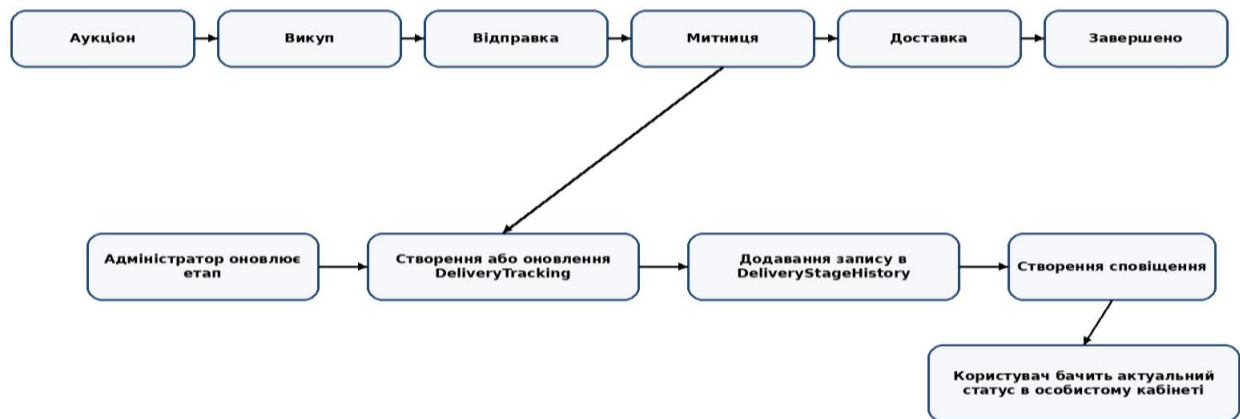


Рисунок 2.5 – Алгоритм супроводу доставки автомобіля під замовлення

Алгоритм зміни статусу автомобіля реалізовано окремим маршрутом PATCH /api/cars/{id}/status. Адміністратор вибирає новий стан, наприклад «в наявності», «під замовлення», «в дорозі», «зарезервовано» або «продано». Після перевірки прав доступу система порівнює новий статус із поточним, оновлює основний запис автомобіля та створює запис CarStatusHistory із попереднім і новим значенням, коментарем, датою та користувачем, який виконав зміну. Додатково статус автомобіля може змінюватися автоматично під час опрацювання заявки, наприклад при схваленні або завершенні замовлення.

Алгоритм зміни ціни виконується під час редагування картки автомобіля маршрутом PUT /api/cars/{id}. Серверна частина порівнює нове значення вартості з попереднім. Якщо ціна відрізняється, у системі створюється запис CarPriceHistory, де фіксуються старе та нове значення, валюта, дата зміни та користувач, що вніс коригування. Завдяки цьому адміністратор може переглядати динаміку ціни і контролювати історію змін без втрати комерційно важливої інформації.

Структура API поділяється на функціональні групи відповідно до модулів проєкту. За реєстрацію та автентифікацію відповідають RegisterController і JWT-механізм входу, за профіль — ProfileController, за каталог автомобілів —

CarController, CarImageController та CarHistoryController. Робота із заявками реалізована в CarOrderController, доставка — у DeliveryTrackingController, персональні функції — у FavoriteCarController, SavedCarSearchController і NotificationController, а блогові можливості — у PostController, CategoryController, CommentController та PostLikeController. Такий поділ спрощує супровід коду та дає змогу розширювати систему без суттєвої зміни вже реалізованих модулів.

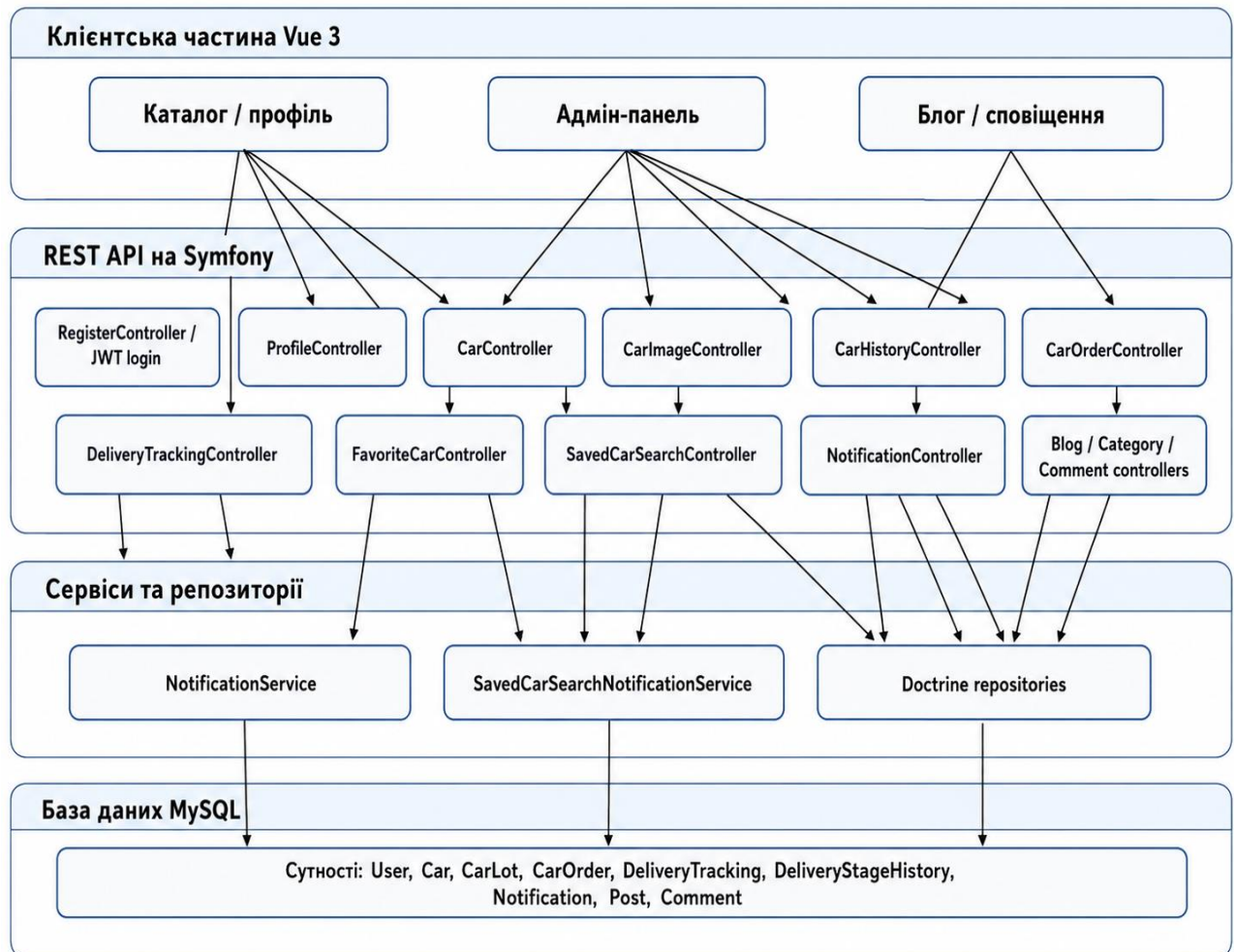


Рисунок 2.6 – Структура взаємодії основних API-контролерів

Динамічна взаємодія між клієнтською частиною, API, репозиторієм і базою даних під час оформлення заявки подана у вигляді діаграми послідовності. Вона деталізує не тільки факт створення заявки, а й перевірку авторизації, валідацію контактних даних, пошук дублю заявки та запис результату до бази даних.

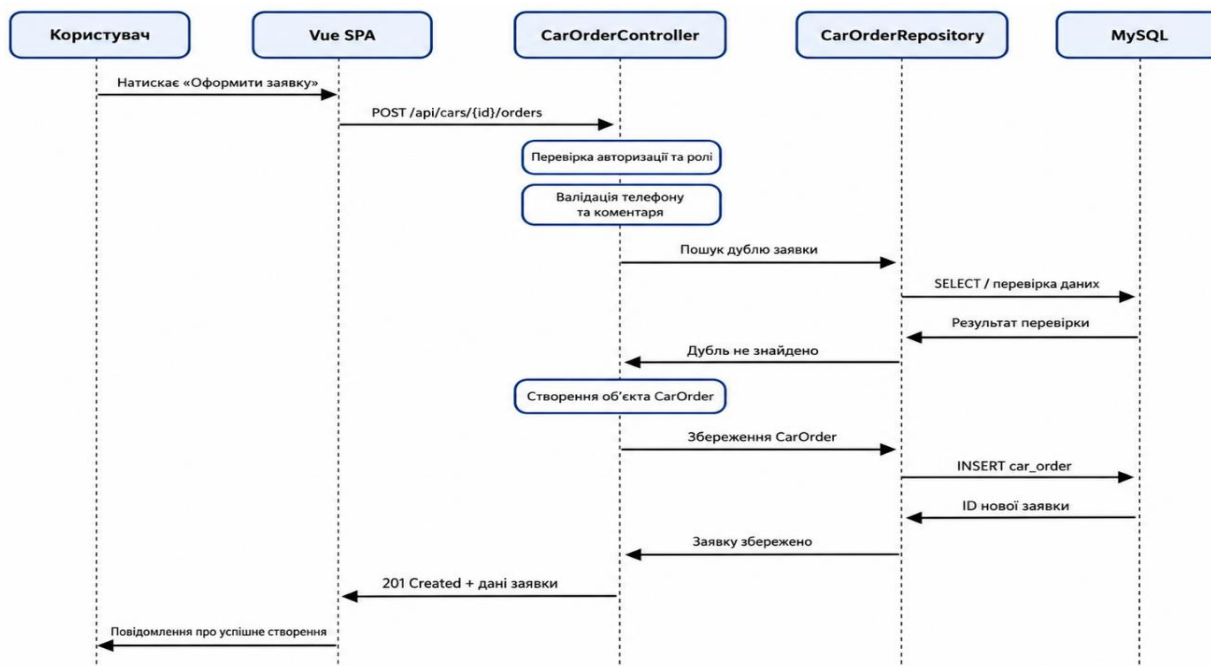


Рисунок 2.7 – Діаграма послідовності процесу оформлення заявки на автомобіль

На діаграмі послідовності показано, що користувач ініціює заявку з інтерфейсу Vue, після чого клієнтська частина надсилає POST-запит до CarOrderController. Сервер перевіряє автентифікацію, доступність автомобіля, коректність номера телефону та відсутність уже створеної заявки на цей автомобіль. Після успішної перевірки створюється запис CarOrder, а клієнтська частина отримує відповідь зі статусом 201 і відображає повідомлення про успішне створення заявки.

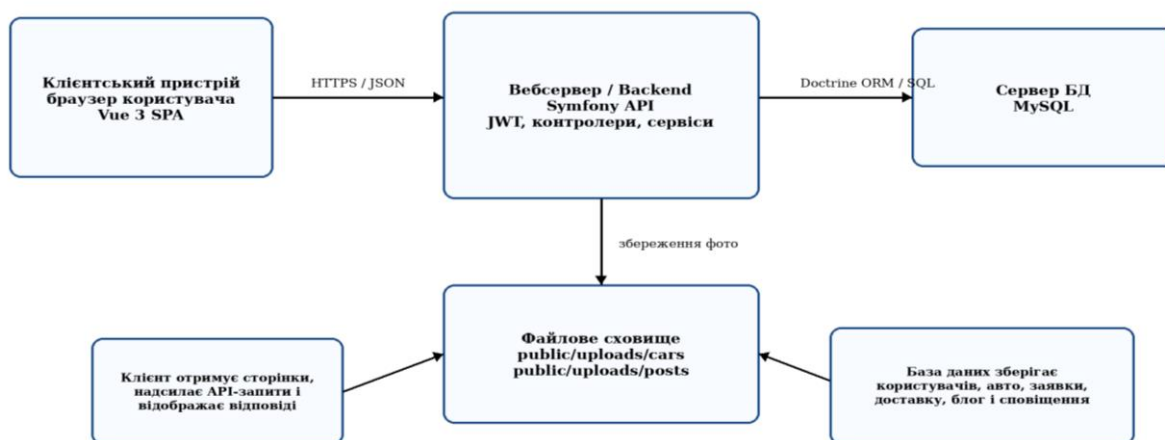


Рисунок 2.8 – Діаграма розгортання вебплатформи

Діаграма розгортання відображає фізичну організацію роботи системи. Користувач працює з вебінтерфейсом у браузері, клієнтська частина звертається до Symfony API через HTTP-запити, серверна частина обробляє бізнес-логіку, а дані зберігаються у MySQL через Doctrine ORM. Окремо передбачено файлове сховище для зображень автомобілів і публікацій блогу, що дає змогу не перевантажувати базу даних великими медіафайлами.

Таблиця 2.4

Основні групи API-маршрутів

Група маршрутів	Призначення	Приклади дій
/api/register, /api/login	Реєстрація та автентифікація користувачів	створення облікового запису, вхід, отримання JWT
/api/me	Робота з профілем користувача	перегляд профілю, редагування даних, зміна пароля
/api/cars	Робота з автомобілями	список, деталі, створення, редагування, видалення, зміна статусу
/api/cars/{id}/images	Керування зображеннями автомобіля	завантаження фото, вибір головного зображення, видалення
/api/cars/{id}/history	Перегляд історії автомобіля	історія ціни, статусів і власників
/api/car-lots	Керування автомобільними майданчиками	список, створення, редагування, видалення локацій
/api/cars/{id}/orders, /api/my-orders	Робота із заявками	створення заявки, перегляд власних заявок
/api/admin/orders	Адміністрування заявок	перегляд усіх заявок, зміна статусу, коментар адміністратора
/api/orders/{id}/tracking	Відстеження доставки	перегляд трекінгу, створення або оновлення доставки, додавання етапів
/api/notifications	Сповіщення користувачів	отримання повідомлень, позначення як прочитане
/api/saved-car-searches	Збережені пошуки	створення пошуку, оновлення параметрів, увімкнення email-повідомлень
/api/posts, /api/categories	Блог та категорії	створення публікацій, перегляд, редагування, коментарі та вподобання
/api/admin/analytics	Аналітичні дані адміністратора	перегляд узагальнених показників системи

Механізм сповіщень побудовано на використанні сутності Notification, сервісу NotificationService та контролера NotificationController. Повідомлення створюються після зміни статусу заявки, оновлення трекінгу доставки, додавання нового етапу доставки, а також у разі спрацювання сервісу SavedCarSearchNotificationService для збережених пошуків. Користувач може переглядати список сповіщень через маршрут /api/notifications, позначати окреме повідомлення як прочитане або змінювати статус усіх сповіщень одразу. Це забезпечує своєчасне інформування про важливі події в системі.



Рисунок 2.9 – Схема створення та відображення сповіщень

2.4. Реалізація застосунку вебплатформи

Реалізація застосунку виконана відповідно до спроектованої архітектури та фактичної структури проєкту. Серверний застосунок містить API-контролери для авторизації, профілю, автомобілів, майданчиків, зображень, історії змін, заявок, доставки, сповіщень, збережених пошуків, блогу, коментарів і аналітики. Клієнтська частина містить сторінки каталогу, детальної інформації про автомобіль, особистого кабінету, адміністративної панелі та блогу. Взаємодія між ними здійснюється через REST-запити, а захищені операції виконуються лише після перевірки JWT-токена та ролі користувача.

Реалізація функціоналу вебплатформи виконується відповідно до клієнт-серверної архітектури. Серверна частина на Symfony відповідає за маршрути API, перевірку прав доступу, роботу з базою даних, валідацію даних і виконання бізнес-логіки. Клієнтська частина на Vue відповідає за відображення інтерфейсу, роботу з формами, взаємодію з API та оновлення сторінок відповідно до відповідей сервера.

Модуль автомобілів є основним у системі. Він забезпечує отримання списку автомобілів, перегляд детальної інформації, створення нових записів, редагування параметрів, видалення автомобіля, зміну статусу та пошук схожих пропозицій. Під час створення або редагування автомобіля адміністратор заповнює технічні характеристики, комерційні дані, опис і локацію. Окремий модуль зображень дозволяє додавати фотографії, позначати головне фото та видаляти зайві зображення.

Модуль автомобільних майданчиків дозволяє адміністратору працювати з локаціями. Для кожного майданчика можна зберігати назву, місто, адресу, телефон, графік роботи та опис. Це дає змогу не обмежувати систему одним складом або офісом і робить її придатною для компанії з декількома філіями.

Модуль заявок реалізує процес взаємодії користувача з адміністрацією. Користувач створює заявку на конкретний автомобіль, після чого вона потрапляє до адміністративної панелі. Адміністратор переглядає контактні дані, коментар клієнта, автомобіль, змінює статус заявки та може залишити службовий коментар. Якщо заявка пов'язана з автомобілем під замовлення, для неї може бути створений трекінг доставки.

Модуль доставки забезпечує поетапне відстеження автомобіля. Поточний етап зберігається у DeliveryTracking, а зміни етапів фіксуються в DeliveryStageHistory. Така реалізація дозволяє не тільки показати останній стан, а й відновити історію руху автомобіля. Користувач бачить цю інформацію в особистому кабінеті, а адміністратор оновлює її через службовий інтерфейс.

Модуль історії автомобіля підвищує контрольованість даних. Під час зміни ціни, статусу або власника створюються окремі історичні записи. Завдяки цьому адміністратор може перевірити, коли й чому була змінена ціна, який статус автомобіль мав раніше та хто був відповідальним за певний період. Це особливо корисно для автомобільних майданчиків, де один транспортний засіб може проходити кілька етапів продажу або доставки.

Персональні модулі користувача включають обране, збережені пошуки та сповіщення. Обране дає змогу швидко повернутися до цікавих пропозицій,

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		39

збережені пошуки дозволяють повторно застосовувати складні фільтри, а сповіщення інформують про зміни, пов'язані із заявками або доставкою. Разом ці функції роблять платформу більш зручною для покупця та наближують її до сучасних автомобільних маркетплейсів.

Блог винесено в окремий функціональний розділ. У ньому адміністратор може створювати публікації, додавати зображення, прив'язувати записи до категорій і працювати з коментарями. Користувачі можуть переглядати матеріали, залишати коментарі та ставити вподобання. Такий розділ дозволяє зберегти початкову ідею автоблогу, але перенести її в окрему частину вебплатформи для автомобільних майданчиків.

Адміністративна аналітика використовується для узагальненого перегляду стану системи. Вона може відображати кількість автомобілів за статусами, кількість заявок, активність користувачів, популярні пропозиції та інші показники. Це допомагає адміністратору оцінювати роботу майданчика та своєчасно звертати увагу на проблемні процеси.

Таблиця 2.5

Розмежування доступу користувачів

Функція	Гість	Користувач	Адміністратор
Перегляд каталогу	Так	Так	Так
Пошук фільтрація автомобілів	Так	Так	Так
Перегляд детальної сторінки	Так	Так	Так
Додавання до обраного	Ні	Так	Так
Збереження пошуків	Ні	Так	Так
Оформлення заявки	Ні	Так	Так
Перегляд власних заявок	Ні	Так	Так
Перегляд сповіщень	Ні	Так	Так
Керування автомобілями	Ні	Ні	Так
Керування зображеннями	Ні	Ні	Так
Керування майданчиками	Ні	Ні	Так
Керування доставкою	Ні	Ні	Так
Перегляд історії змін	Ні	Ні	Так
Керування блогом	Ні	Ні	Так
Перегляд аналітики	Ні	Ні	Так

Реалізований функціонал відповідає модульній структурі, описаній у проєктному розділі. Кожен модуль має власну зону відповідальності, а взаємодія між ними відбувається через API, сутності та сервіси. Такий підхід спрощує подальше вдосконалення платформи, наприклад додавання нових ролей, розширення аналітики або інтеграцію з зовнішніми сервісами повідомлень.

Блог у реалізації відокремлений від каталогу автомобілів, але використовує спільну систему користувачів і ролей. Адміністратор може керувати публікаціями та категоріями, а користувачі можуть читати матеріали, залишати коментарі й реагувати на записи. Це дозволяє платформі виконувати не тільки комерційну, а й інформаційну функцію.

Сповіщення реалізовані як допоміжний сервіс, який реагує на важливі події системи. Повідомлення може створюватися після появи нової заявки, зміни її статусу, оновлення доставки або спрацювання збереженого пошуку. Завдяки цьому користувач і адміністратор отримують інформацію в межах платформи, а не лише через зовнішні канали зв'язку.

Реалізація доставки тісно пов'язана із заявками. Для кожної заявки може бути створений поточний запис доставки, у якому зберігається актуальний етап, орієнтовна дата та примітка. Окремо фіксується історія етапів, що дозволяє відновити послідовність руху автомобіля. Такий механізм корисний для автомобілів під замовлення, оскільки користувач бачить не лише кінцевий результат, а й проміжний стан.

У модулі заявок передбачено перевірку вхідних даних. Перед створенням заявки система перевіряє наявність автомобіля, авторизацію користувача, коректність телефону, довжину коментаря та відсутність дублю заявки. Це дає змогу не створювати зайві записи й підтримувати чистоту даних у базі. Якщо перевірка не проходить, користувач отримує повідомлення про помилку.

Модуль зображень реалізований окремо від основної логіки автомобіля. Це дозволяє додавати кілька фотографій до одного транспортного засобу, змінювати головне зображення та видаляти файли без редагування всіх параметрів

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		41

автомобіля. Після завантаження файл зберігається в каталозі проєкту, а в базі даних фіксується зв'язок між записом автомобіля та відповідним зображенням.

Форма створення або редагування автомобіля реалізована так, щоб адміністратор міг послідовно заповнити технічні, комерційні та службові дані. До технічних параметрів належать марка, модель, рік, пробіг, двигун, коробка передач, привід, тип кузова та колір. До комерційних параметрів належать ціна, валюта, статус, опис і локація. Розділення даних у формі спрощує роботу адміністратора та зменшує ризик пропуску важливого поля.

Під час реалізації каталогу особлива увага приділена роботі з фільтрами. Клієнтська частина формує параметри пошуку на основі введених користувачем значень, а серверна частина обробляє їх у межах запиту до списку автомобілів. Такий підхід дозволяє не завантажувати на клієнт усі записи одразу, а передавати лише ті автомобілі, які відповідають обраним умовам. Це важливо для масштабованості каталогу і зменшення часу відгуку сторінки.

2.5. Об'єктно-орієнтована та компонентна будова програмного застосунку

Об'єктно-орієнтована модель вебплатформи побудована навколо сутностей, які відображають реальні об'єкти предметної області автомобільного майданчика. Центральне місце займає сутність Car, оскільки саме автомобіль є основним ресурсом каталогу, предметом заявки користувача, об'єктом доставки та джерелом історичних змін. Навколо цієї сутності формується група допоміжних класів, що забезпечують збереження фотографій, статусів, цінових змін, належності до майданчика, зв'язку з користувачами та участі в замовленнях. Такий підхід дозволяє не перевантажувати одну таблицю зайвими службовими даними, а розподілити інформацію між пов'язаними об'єктами.

Сутність User використовується для зберігання облікових записів і ролей користувачів. Вона пов'язана з публікаціями блогу, коментарями, вподобаннями, заявками, обраними автомобілями, збереженими пошуками та сповіщеннями. У системі передбачено рольове розмежування доступу, тому той самий клас користувача може представляти як звичайного покупця, так і

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		42

адміністратора. Різниця між цими ролями не дублюється в окремих таблицях, а визначається через набір ролей у полі roles, що спрощує автентифікацію, авторизацію та перевірку доступу в контролерах.

Клас Car містить технічні та комерційні характеристики автомобіля. До таких характеристик належать марка, модель, рік випуску, VIN-код, пробіг, тип пального, коробка передач, тип приводу, кузов, колір, об'єм двигуна, потужність, ціна, валюта, дата надходження, статус, місце розташування та опис. У межах об'єктної моделі автомобіль також має зв'язок із майданчиком CarLot, колекцією зображень CarImage, історією цін CarPriceHistory, історією статусів CarStatusHistory, історією власників CarOwnerHistory і заявками CarOrder. Така деталізація дозволяє підтримувати повний життєвий цикл транспортного засобу в межах платформи.

Сутність CarLot описує автомобільний майданчик або філію. Вона дає змогу не обмежувати систему одним складом чи однією локацією. Для кожного майданчика зберігаються назва, місто, адреса, контактний номер, графік роботи та опис. Завдяки зв'язку між CarLot і Car адміністратор може розподіляти автомобілі між різними локаціями, а користувач отримує зрозумілу інформацію про те, де саме знаходиться автомобіль, який його зацікавив. Така модель є важливою для масштабування системи, оскільки у майбутньому вона може підтримувати декілька майданчиків у різних містах.

Для роботи із заявками використовується сутність CarOrder. Вона пов'язує користувача з конкретним автомобілем і зберігає стан звернення клієнта. У заявці фіксується номер телефону, коментар користувача, коментар адміністратора, поточний статус і дата оновлення. Така структура дозволяє відокремити факт зацікавленості користувача від самого автомобіля. Автомобіль може існувати в каталозі незалежно від заявок, а заявки можуть оброблятися окремо в адміністративній панелі. У разі зміни статусу заявки система може автоматично змінювати стан автомобіля, наприклад переводити його в зарезервований або проданий стан.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		43

Модуль доставки побудований на двох пов'язаних сутностях: DeliveryTracking і DeliveryStageHistory. Перша зберігає поточний стан доставки для конкретної заявки, а друга фіксує історію зміни етапів. Такий поділ потрібний тому, що користувачеві важливо бачити актуальний етап, але адміністратору також потрібна хронологія попередніх дій. Наприклад, автомобіль може послідовно проходити етапи аукціону, викупу, відправки, митниці, доставки та завершення. Кожне оновлення зберігається окремим записом із датою, статусом, описом і користувачем, який виконав зміну.

Історичні сутності CarPriceHistory, CarStatusHistory і CarOwnerHistory застосовуються для аудиту змін. Вони не замінюють основний запис автомобіля, а доповнюють його службовою інформацією. Під час редагування ціни створюється запис із попереднім і новим значенням, під час зміни статусу зберігаються старий і новий стани, а під час зміни власника або відповідальної особи формується окрема історична подія. Така модель підвищує контрольованість системи та дозволяє адміністратору перевірити, які зміни відбувалися з автомобілем упродовж його перебування на майданчику.

Персональні функції користувача реалізовані через сутності FavoriteCar, SavedCarSearch і Notification. FavoriteCar зберігає зв'язок між користувачем та автомобілем, який додано до обраного. SavedCarSearch дає змогу зберігати параметри фільтрації каталогу, щоб користувач міг повторно застосувати складний пошук без ручного введення всіх параметрів. Notification використовується для інформування про події, що мають значення для користувача або адміністратора. До таких подій належать створення заявки, зміна її статусу, оновлення доставки або поява автомобіля, що відповідає збереженому пошуку.

Блогова частина платформи представлена сутностями Post, Category, Comment і PostLike. Вона винесена в окремий логічний блок, але використовує ту саму систему користувачів і ролей. Адміністратор може створювати публікації, призначати категорії та керувати коментарями, а користувачі можуть переглядати матеріали, залишати коментарі та вподобання. Наявність блогу

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		44

розширює платформу від звичайного каталогу до інформаційного ресурсу, де можна публікувати огляди автомобілів, поради щодо вибору, новини майданчика та матеріали про доставку авто під замовлення.

Серверна частина проєкту має шарову будову. Зовнішній шар утворюють API-контролери, які приймають HTTP-запити, перевіряють вхідні параметри, викликають потрібні сервіси або репозиторії та формують JSON-відповіді для клієнтської частини. Контролери не повинні містити надмірної бізнес-логіки, оскільки їхнє основне призначення полягає в маршрутизації запитів і підготовці відповіді. У розробленій системі окремі контролери відповідають за автомобілі, зображення, історію, заявки, доставку, сповіщення, профіль, блог, коментарі, категорії, обране та збережені пошуки. Така організація зменшує зв'язаність модулів і відповідає підходам до побудови підтримуваного корпоративного застосунку [25; 26].

Шар сервісів використовується для операцій, які можуть викликатися з різних частин системи або мають окреме бізнес-призначення. Наприклад, NotificationService відповідає за створення повідомлень після важливих подій, а SavedCarSearchNotificationService використовується для перевірки збережених пошуків і формування сповіщень про відповідні автомобілі. Винесення такої логіки в сервіси зменшує залежність контролерів один від одного та спрощує подальше тестування окремих частин системи.

Репозиторії Doctrine утворюють шар доступу до даних. Вони відповідають за пошук, фільтрацію, сортування, пагінацію та вибірку сутностей із бази даних. Наприклад, репозиторій автомобілів виконує пошук за параметрами каталогу, а репозиторій заявок повертає списки звернень для користувача або адміністратора. Завдяки репозиторіям логіка формування складних запитів не розміщується безпосередньо в контролерах, що робить код більш зрозумілим і придатним до супроводу.

Клієнтська частина Vue також побудована за компонентним принципом. Основні сторінки реалізовано як окремі представлення, а повторювані частини інтерфейсу винесено в компоненти. Наприклад, картка автомобіля, фільтри

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		45

каталогу, бейдж статусу, пагінація, блок сповіщень, форма заявки та елемент навігації можуть використовуватися на кількох сторінках. Така організація дозволяє уникнути дублювання коду та спрощує внесення змін до інтерфейсу. Якщо змінюється вигляд картки автомобіля, достатньо оновити один компонент, після чого зміна відобразиться в усіх місцях його використання. Компонентна структура клієнтської частини пов'язана з використанням HTML, JavaScript і повторно використовуваних елементів інтерфейсу [8].

Взаємодія між клієнтською та серверною частинами відбувається через REST-запити. Після завантаження сторінки Vue-компонент виконує запит до потрібного маршруту API, отримує відповідь у форматі JSON і відображає дані в інтерфейсі. Для захищених маршрутів у заголовку запиту передається JWT-токен. Сервер перевіряє токен, визначає користувача та його роль, після чого або виконує операцію, або повертає відповідь про відсутність доступу. Такий механізм використовується для адміністративної панелі, особистого кабінету, роботи із заявками, доставкою, обраним і сповіщеннями. Передавання даних через HTTP-клієнт і використання токена автентифікації узгоджуються з принципами HTTP, REST і JWT [11; 21].

Під час проєктування компонентної будови важливо було забезпечити незалежність модулів. Каталог автомобілів може працювати без блогу, блог може розвиватися окремо від модуля доставки, а система сповіщень може використовуватися різними бізнес-процесами. Така слабка зв'язаність між модулями підвищує підтримуваність програмного продукту. У разі додавання нового функціоналу, наприклад фінансового модуля, розрахунку вартості доставки або інтеграції із зовнішнім сервісом перевірки VIN, не потрібно повністю змінювати вже реалізовані частини системи. Така модульність узгоджується з підходами шаблонів проєктування та принципами підтримуваного коду [24; 25].

Окрему увагу приділено обробці помилок і перевірці даних. Серверна частина перевіряє формат JSON-запиту, наявність обов'язкових полів, коректність номера телефону, допустимість статусів, існування автомобіля або

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		46

заявки та права доступу користувача. Клієнтська частина отримує повідомлення про помилки та відображає їх у зрозумілій формі. Такий підхід зменшує ризик створення некоректних записів у базі даних і покращує користувацький досвід.

Компонентна будова також враховує майбутнє розширення платформи. У перспективі до системи можна додати модуль фінансових розрахунків, інтеграцію з платіжними сервісами, розширену аналітику, автоматичне формування договорів, інтеграцію з CRM або сервісами доставки. Завдяки поділу на сутності, контролери, сервіси, репозиторії та Vue-компоненти такі зміни можна виконувати поступово, не порушуючи базову логіку каталогу, заявок і доставки.

Таким чином, об'єктно-орієнтована та компонентна будова застосунку забезпечує логічне відображення предметної області в програмному коді. Сутності описують дані, контролери організовують доступ через API, сервіси реалізують повторно використовувану бізнес-логіку, репозиторії відповідають за роботу з базою даних, а Vue-компоненти формують інтерфейс користувача. Такий підхід відповідає вимогам до інженерного проектування програмних систем і створює основу для подальшого розвитку вебплатформи.

Під час розробки серверної частини було важливо забезпечити єдину логіку відповідей API. Для успішних операцій сервер повертає дані ресурсу або повідомлення про виконану дію, а для помилкових ситуацій формується відповідь із кодом стану та описом проблеми. Наприклад, у разі відсутності авторизації повертається відповідь з кодом 401, у разі відсутності прав адміністратора — 403, якщо ресурс не знайдено — 404, а при некоректних вхідних даних — 422. Така уніфікація дозволяє клієнтській частині однаково обробляти помилки в різних модулях і показувати користувачеві зрозумілі повідомлення.

Передавання даних між Vue та Symfony організовано таким чином, щоб клієнтська частина не працювала безпосередньо з базою даних. Усі операції виконуються тільки через API-маршрути. Наприклад, під час відкриття каталогу Vue надсилає запит до маршруту `/api/cars`, а під час створення заявки

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		47

використовує маршрут `/api/cars/{id}/orders`. Сервер перевіряє дані, звертається до репозиторіїв і повертає відповідь у форматі JSON. Завдяки цьому клієнтська частина залишається незалежною від внутрішньої структури бази даних і може змінювати інтерфейс без прямого впливу на схему збереження інформації.

Розмежування доступу реалізовано на рівні контролерів і бізнес-логіки. Публічні маршрути, такі як перегляд каталогу, деталей автомобіля або записів блогу, доступні без адміністративних прав. Дії, що змінюють службові дані, наприклад створення автомобіля, редагування статусу, оновлення доставки або керування коментарями, виконуються лише після перевірки ролі адміністратора. Такий підхід захищає систему від випадкового або навмисного втручання звичайних користувачів у внутрішні процеси майданчика. Такий механізм враховує базові рекомендації щодо захисту вебзастосунків і перевірки прав доступу [17; 18].

У клієнтській частині важливу роль відіграє маршрутизація. Окремі маршрути відповідають за головну сторінку, каталог, сторінку автомобіля, блог, профіль користувача, обране, порівняння, збережені пошуки, заявки та адміністративні розділи. Під час переходу між сторінками не відбувається повне перезавантаження вебзастосунку, тому інтерфейс працює як односторінковий застосунок. Це підвищує швидкість взаємодії та створює зручніший користувацький досвід. Побудова навігації реалізується засобами Vue Router [9].

Компоненти Vue поділяються за призначенням. Частина компонентів відповідає за відображення даних, наприклад картки автомобілів, таблиці заявок або блоки сповіщень. Інші компоненти відповідають за введення даних, наприклад форми авторизації, фільтри каталогу, форми створення автомобіля та форми оновлення доставки. Таке розділення спрощує валідацію, повторне використання елементів і підтримку інтерфейсу. Якщо потрібно змінити вигляд фільтрації або додати новий параметр пошуку, зміни вносяться в окремий компонент і відповідний API-запит.

Архітектура даних також враховує необхідність підтримки зображень. Замість зберігання фотографій безпосередньо в таблиці автомобіля

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		48

використовується окрема сутність CarImage. Це дає змогу додавати до одного автомобіля декілька зображень, визначати головне фото та видаляти окремі файли без зміни основного запису автомобіля. Такий підхід є практичним для каталогу, оскільки автомобільна пропозиція зазвичай містить кілька фотографій із різних ракурсів.

Для адміністратора важливо мати не лише можливість створювати записи, а й контролювати їх актуальність. Тому адміністративна панель працює як окремий набір сторінок, що звертаються до службових API-маршрутів. Через неї виконується керування автомобілями, заявками, доставкою, блогом, коментарями та аналітичними показниками. Усі ці операції пов'язані з роллю адміністратора, що дозволяє відокремити робочу частину системи від публічного інтерфейсу покупця.

Важливою архітектурною особливістю є поділ операцій читання та зміни даних. Запити на отримання інформації переважно виконуються методами GET, тоді як створення, редагування та видалення використовують POST, PUT, PATCH або DELETE. Такий підхід відповідає принципам REST і робить структуру API зрозумілою. Наприклад, перегляд автомобіля не змінює стан системи, а оновлення статусу або створення заявки завжди виконується окремим запитом, який проходить перевірку прав і валідацію. Таке використання HTTP-методів відповідає принципам REST і семантиці протоколу HTTP [15; 16].

Під час реалізації було враховано можливість подальшого тестування окремих модулів. Оскільки контролери, сервіси, репозиторії та сутності мають різні зони відповідальності, перевірку можна проводити не тільки через інтерфейс, а й на рівні API. Це дозволяє окремо перевіряти створення заявки, зміну статусу, оновлення доставки, отримання сповіщень і фільтрацію каталогу. Такий підхід підвищує надійність програмного продукту та спрощує пошук помилок.

Загалом компонентна та об'єктно-орієнтована модель забезпечує логічний зв'язок між функціональними вимогами й фактичною реалізацією. Кожна сутність відповідає конкретному елементу предметної області, кожен контролер

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		49

обслуговує визначену групу API-маршрутів, а кожен модуль клієнтської частини відповідає за окремий сценарій користувача або адміністратора. Завдяки цьому вебплатформа має зрозумілу архітектуру, яку можна супроводжувати, тестувати та розширювати після завершення дипломного проєкту.

Висновки до другого розділу

У другому розділі було виконано проєктування вебплатформи для автомобільних майданчиків. Було визначено функціональні та нефункціональні вимоги, описано групи користувачів і встановлено, які можливості повинні бути доступні гостю, зареєстрованому користувачу та адміністратору.

Окрему увагу приділено інформаційній моделі бази даних. Було визначено основні сутності системи, серед яких User, Car, CarLot, CarImage, CarOrder, DeliveryTracking, DeliveryStageHistory, CarPriceHistory, CarStatusHistory, CarOwnerHistory, FavoriteCar, SavedCarSearch, Notification, Post, Category і Comment. Запропонована структура забезпечує зберігання не тільки основних даних автомобіля, а й історії змін, заявок, доставки та персональних дій користувачів.

Також у розділі було описано алгоритми пошуку та фільтрації автомобілів, оформлення заявки, супроводу доставки, зміни статусу, зміни ціни, створення сповіщень і роботи API. Описана структура відповідає фактичним модулям проєкту та створює основу для подальшого опису інтерфейсу, реалізації й тестування вебплатформи.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		50

РОЗДІЛ 3. ІНТЕРФЕЙС ТА ПОРЯДОК РОБОТИ З ВЕБПЛАТФОРМОЮ ДЛЯ АВТОМОБІЛЬНИХ МАЙДАНЧИКІВ

3.1. Порядок встановлення та налаштування параметрів системи

Для запуску вебплатформи необхідно підготувати серверну та клієнтську частини проєкту. Серверна частина Symfony потребує встановленого PHP, Composer, MySQL і налаштованого файлу середовища .env, у якому вказується підключення до бази даних, параметри JWT-автентифікації та шляхи для збереження зображень. Після встановлення залежностей виконується створення бази даних, запуск міграцій Doctrine та підготовка початкових даних, необхідних для роботи адміністративного облікового запису й базових довідників. Налаштування серверної частини спирається на документацію PHP, Symfony, Doctrine, MySQL і JWT-автентифікації [5; 6].

Клієнтська частина Vue запускається окремо через середовище Node.js. Після встановлення залежностей виконується запуск Vite-сервера розробки, який відкриває інтерфейс вебплатформи в браузері. У налаштуваннях клієнтської частини визначається адреса серверного API, щоб сторінки каталогу, особистого кабінету, адміністративної панелі, блогу та сповіщень могли отримувати дані із Symfony-застосунку. Запуск клієнтської частини пов'язаний із використанням Vue, Vite та Axios для взаємодії з API [8; 10].

Права доступу в системі визначаються ролями користувачів. Звичайний користувач має доступ до каталогу, детальної сторінки автомобіля, обраного, порівняння, власних заявок, трекінгу доставки та сповіщень. Адміністратор отримує доступ до панелі керування, де може створювати автомобілі, завантажувати зображення, змінювати статуси, працювати із заявками, доставкою, блогом, категоріями, коментарями та аналітикою. Такий поділ прав зменшує ризик випадкової або несанкціонованої зміни службових даних.

Після налаштування серверної та клієнтської частин користувач може працювати з вебплатформою через браузер. Подальший опис інтерфейсу подано відповідно до основних ролей системи та сценаріїв використання.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		51

Після налаштування параметрів виконується контрольна перевірка працездатності. Вона включає відкриття каталогу, авторизацію користувача, створення заявки, перегляд сповіщень, вхід до адміністративної панелі та зміну статусу автомобіля. Якщо всі ці дії виконуються без помилок, система вважається підготовленою до подальшого функціонального тестування.

Якщо застосунок розгортається не локально, а на сервері, додатково потрібно налаштувати домен, HTTPS, змінні середовища, права доступу до каталогів і підключення до бази даних. У такому випадку клієнтська частина повинна звертатися не до локальної адреси, а до публічного API. Також важливо переконатися, що сервер дозволяє обробку запитів із потрібного домену.

Під час першого запуску необхідно перевірити, чи відкривається головна сторінка, чи завантажується каталог автомобілів і чи виконується запит до профілю користувача після авторизації. Також потрібно перевірити, чи працює адміністративний вхід, оскільки без нього неможливо протестувати керування автомобілями, заявками, доставкою та блогом.

Після встановлення залежностей запускається серверна частина та окремо запускається клієнтська частина. У режимі розробки Symfony обробляє API-запити, а Vite відкриває інтерфейс у браузері. Такий поділ зручний під час розробки, оскільки дає змогу швидко змінювати інтерфейс і паралельно перевіряти роботу серверної логіки.

Клієнтська частина налаштовується окремо. Після переходу до каталогу Vue-застосунку встановлюються залежності npm. Далі перевіряється адреса API, яку використовує клієнтський код для звернення до Symfony. Якщо адреса вказана неправильно, сторінки можуть завантажуватися, але дані каталогу, профілю, заявок або сповіщень не будуть отримані.

Важливим параметром є каталог для завантаження зображень автомобілів. Він повинен існувати у файловій системі та мати права на запис. Якщо сервер не має доступу до цього каталогу, завантаження фотографій не відбудеться, а автомобіль не зможе отримати повноцінну галерею. Тому перед тестуванням необхідно перевірити структуру директорій і доступність папки для файлів.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		52

Окремого налаштування потребує механізм JWT-автентифікації. Для нього мають бути створені ключі або налаштовані параметри, які дозволяють серверній частині підписувати токени та перевіряти їх під час захищених запитів. Якщо токен не створюється або не передається клієнтською частиною, користувач не зможе отримати доступ до особистого кабінету й адміністративних маршрутів [14].

Для перевірки роботи платформи база даних може бути наповнена тестовими даними. До таких даних належать обліковий запис адміністратора, декілька автомобілів, автомобільні майданчики, категорії блогу та приклади публікацій. Наявність початкових записів дозволяє одразу перевірити каталог, детальні сторінки, фільтрацію, заявки та адміністративну панель.

Після налаштування підключення створюється база даних і застосовуються міграції. Міграції відображають поточну структуру сутностей проєкту та створюють таблиці для користувачів, автомобілів, майданчиків, заявок, доставки, історії змін, сповіщень і блогу. Використання міграцій дозволяє відтворити структуру бази без ручного створення таблиць і зменшує ризик помилок.

Другим етапом є налаштування файлу середовища. У ньому вказується рядок підключення до бази даних, параметри оточення, секрет застосунку та інші службові значення. Для коректної роботи з MySQL необхідно вказати назву бази даних, користувача, пароль, адресу сервера та порт. Помилка в цих параметрах призводить до неможливості створити таблиці або виконати запити до бази.

Першим етапом встановлення є розміщення проєкту в робочому каталозі та встановлення серверних залежностей. Після відкриття каталогу Symfony виконується встановлення пакетів Composer. У результаті формується каталог залежностей, необхідний для роботи контролерів, Doctrine ORM, механізмів безпеки та інших компонентів серверної частини.

Перед запуском вебплатформи необхідно перевірити наявність програмного середовища, потрібного для роботи серверної та клієнтської частин. Серверна частина потребує встановленого PHP, менеджера залежностей

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		53

Composer, системи керування базами даних MySQL і доступу до командного рядка. Клієнтська частина потребує Node.js та npm, оскільки інтерфейс розроблено на основі Vue і збирається через Vite.

3.2. Структура інтерфейсу та порядок роботи з вебплатформою

Користувацький інтерфейс вебплатформи призначений для зручного пошуку автомобілів, перегляду інформації про пропозиції, створення заявок і роботи з особистими функціями. Під час проєктування інтерфейсу враховано, що користувач може відвідувати платформу з різними цілями: швидко знайти автомобіль за параметрами, переглянути актуальні пропозиції, порівняти кілька моделей або простежити стан власного замовлення. Під час опису інтерфейсу враховано загальні підходи до доступності та зручності вебсторінок [20].

Головна сторінка виконує роль початкової точки взаємодії з платформою. На ній розміщено загальний опис сервісу, блок пошуку, посилання на каталог, приклади доступних автомобілів і перехід до інформаційних матеріалів. Завдяки цьому користувач одразу розуміє призначення платформи та може перейти до основного функціоналу.

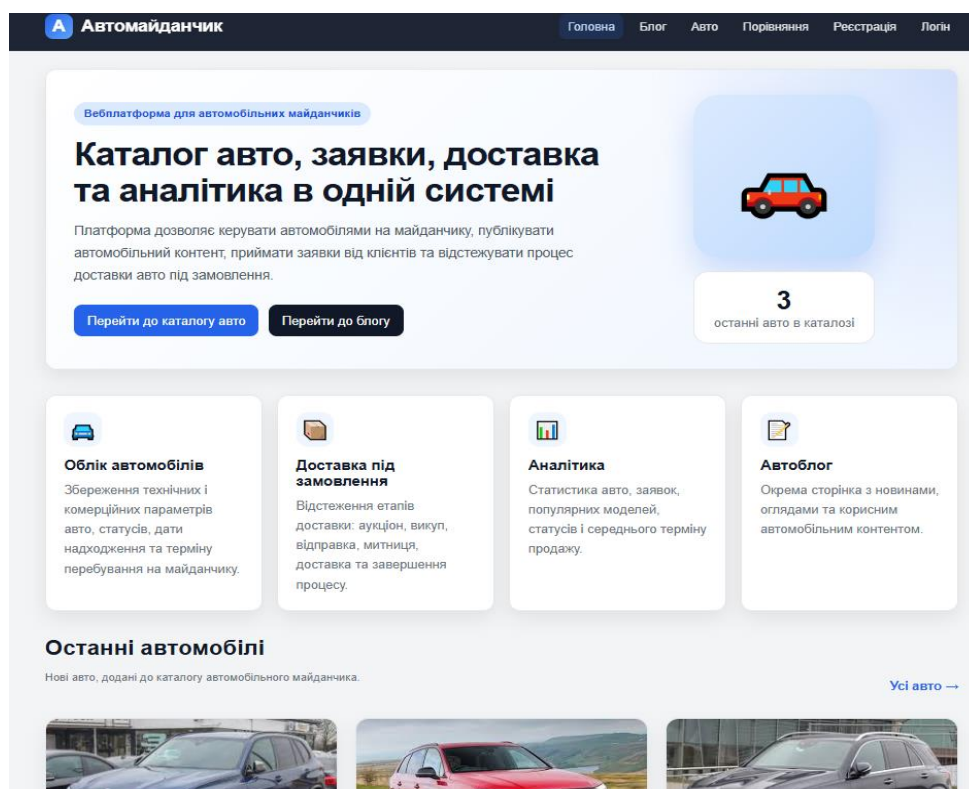


Рисунок 3.1 – Головна сторінка вебплатформи

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		54

Каталог автомобілів є основною сторінкою користувацької частини. У ньому відображаються картки автомобілів, що містять фото, марку, модель, рік випуску, ціну, пробіг і статус. Фільтри дозволяють звужити список за ціною, роком, типом пального, коробкою передач або статусом автомобіля. Такий підхід зменшує час пошуку потрібної пропозиції.

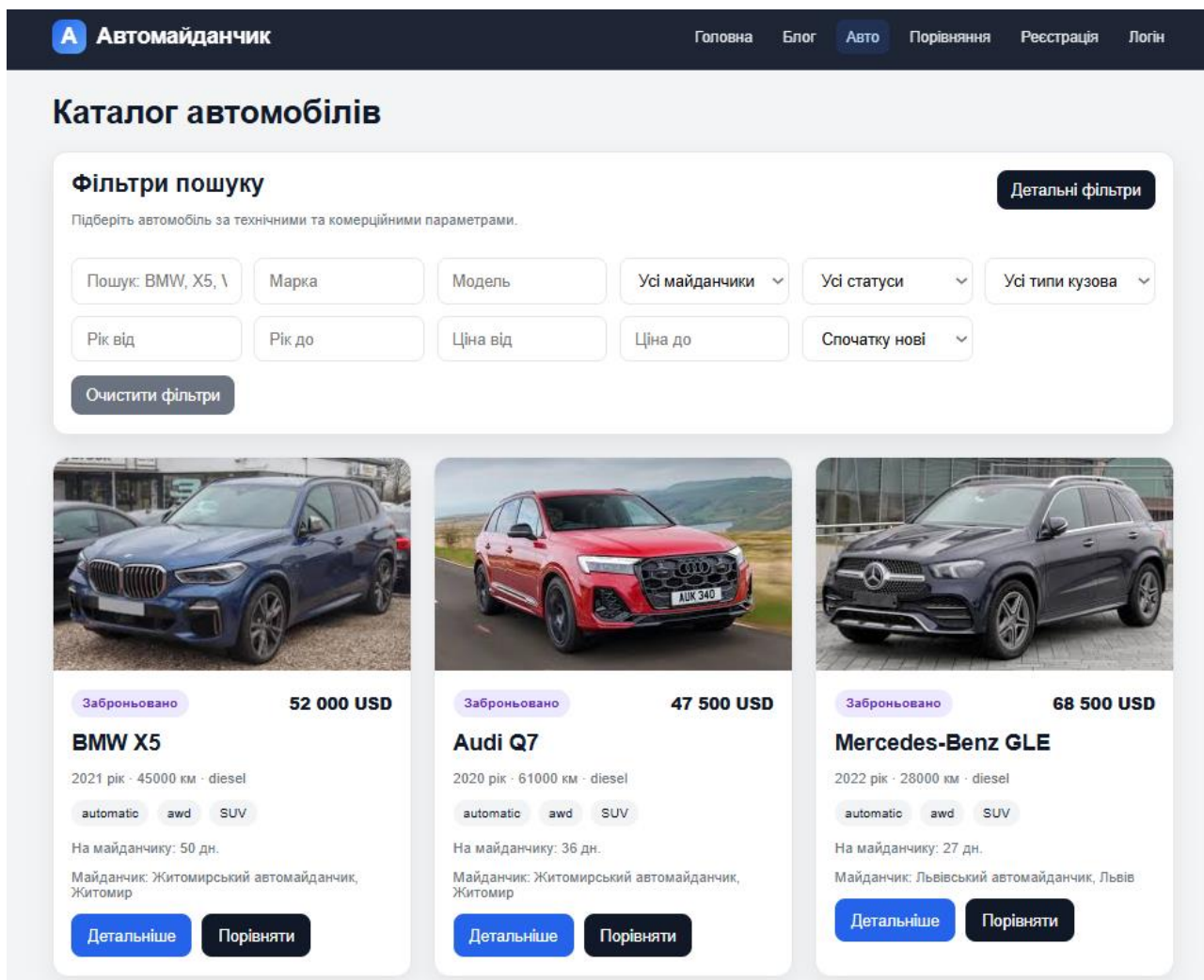


Рисунок 3.2 – Сторінка каталогу автомобілів

Детальна сторінка автомобіля містить повну інформацію про конкретну пропозицію. Користувач може переглянути галерею фотографій, технічні характеристики, комерційні параметри, опис, статус, дату надходження на майданчик і доступні дії. Наявність кнопок заявки, обраного та порівняння робить сторінку не лише інформаційною, а й інтерактивною.

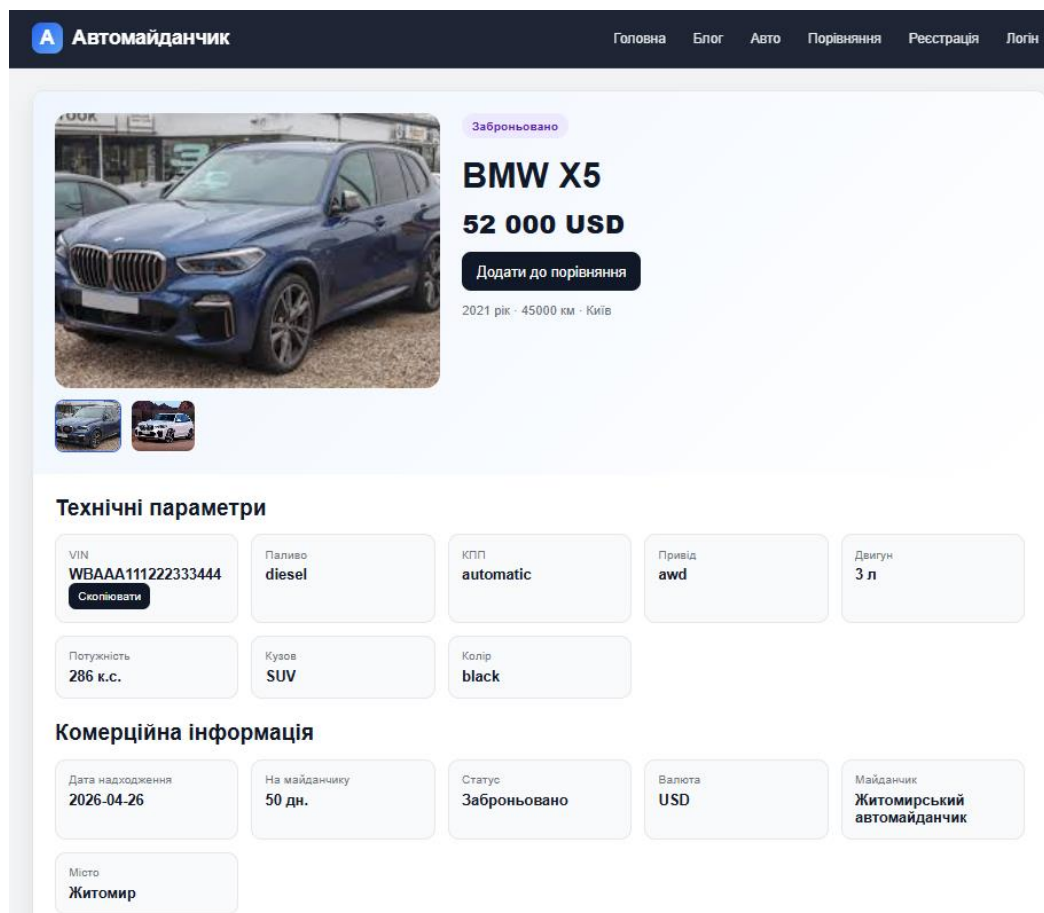


Рисунок 3.3 – Детальна сторінка автомобіля

Авторизація потрібна для доступу до персональних функцій платформи. Після входу користувач отримує можливість створювати заявки, додавати автомобілі до обраного, користуватися порівнянням, переглядати власні замовлення та отримувати сповіщення. Форма авторизації містить мінімально необхідні поля, а сервер перевіряє правильність введених даних.

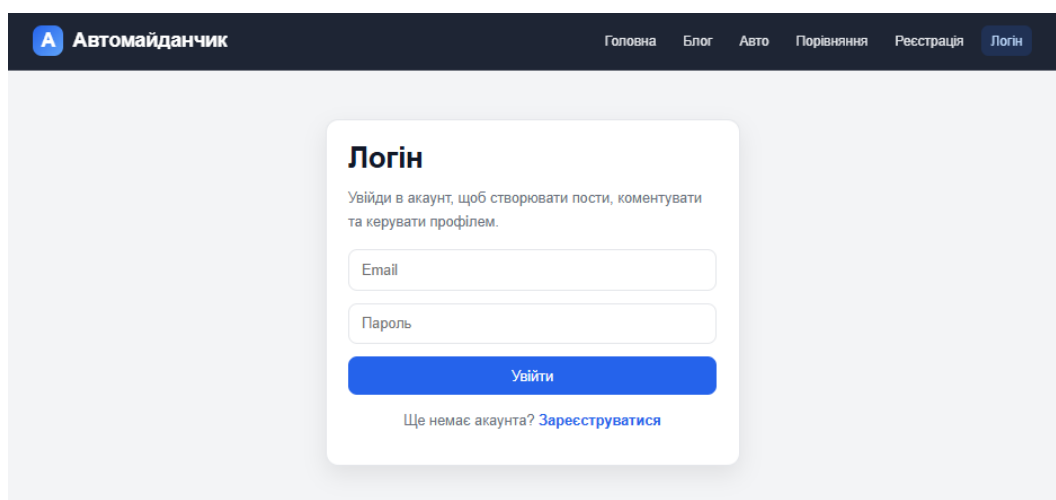


Рисунок 3.4 – Сторінка авторизації користувача

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		56

Особистий кабінет об'єднує персональні можливості користувача. У ньому відображаються заявки, сповіщення, збережені пошуки та інші дані, які пов'язані з обліковим записом. Такий підхід дозволяє не втрачати історію взаємодії з платформою та швидко повертатися до потрібної інформації.

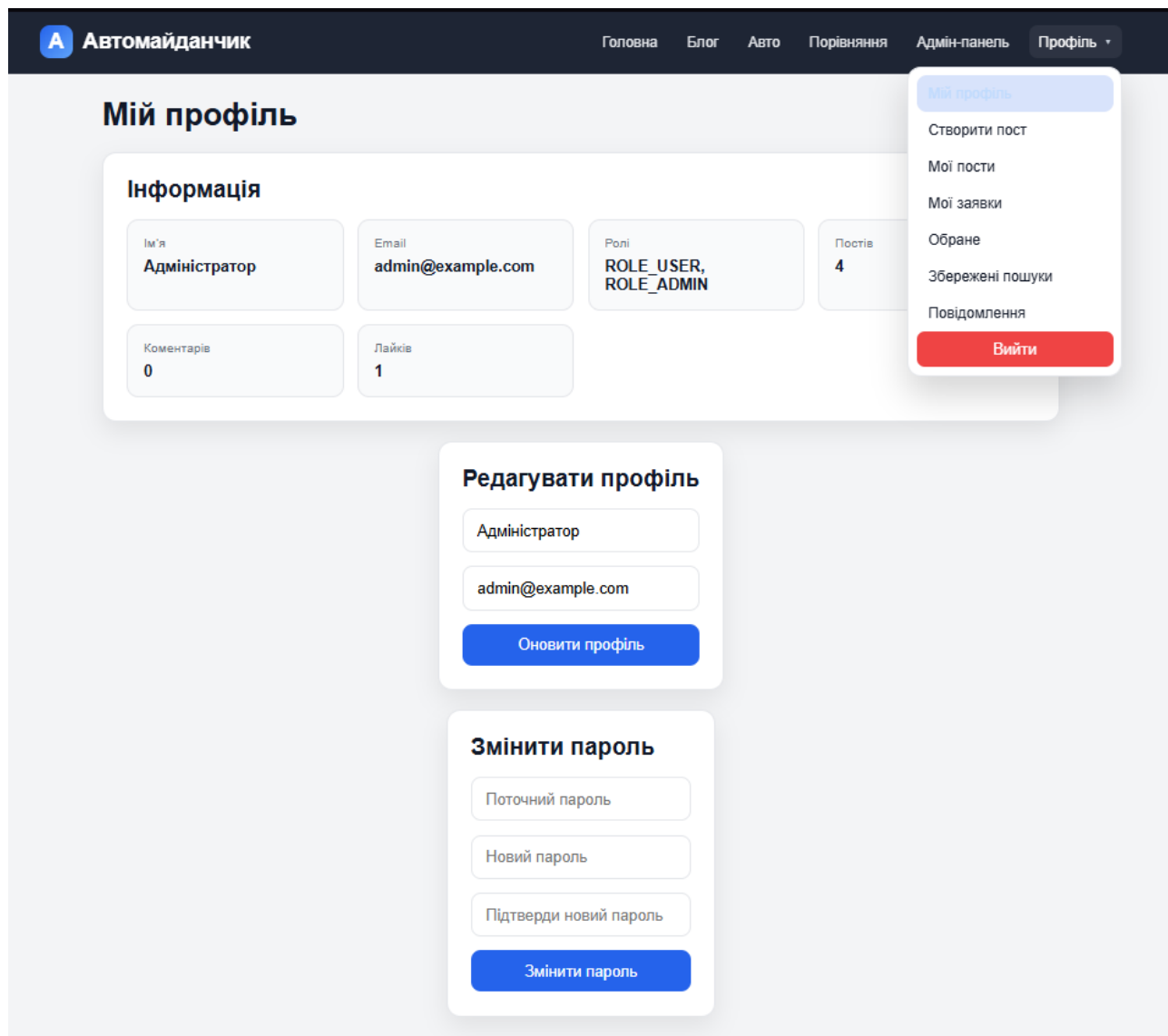


Рисунок 3.5 – Особистий кабінет користувача

Сторінка обраного дає змогу зберігати цікаві автомобілі для повторного перегляду. Користувач може додати авто до списку з каталогу або детальної сторінки, а потім перейти до обраного й порівняти кілька збережених пропозицій між собою.

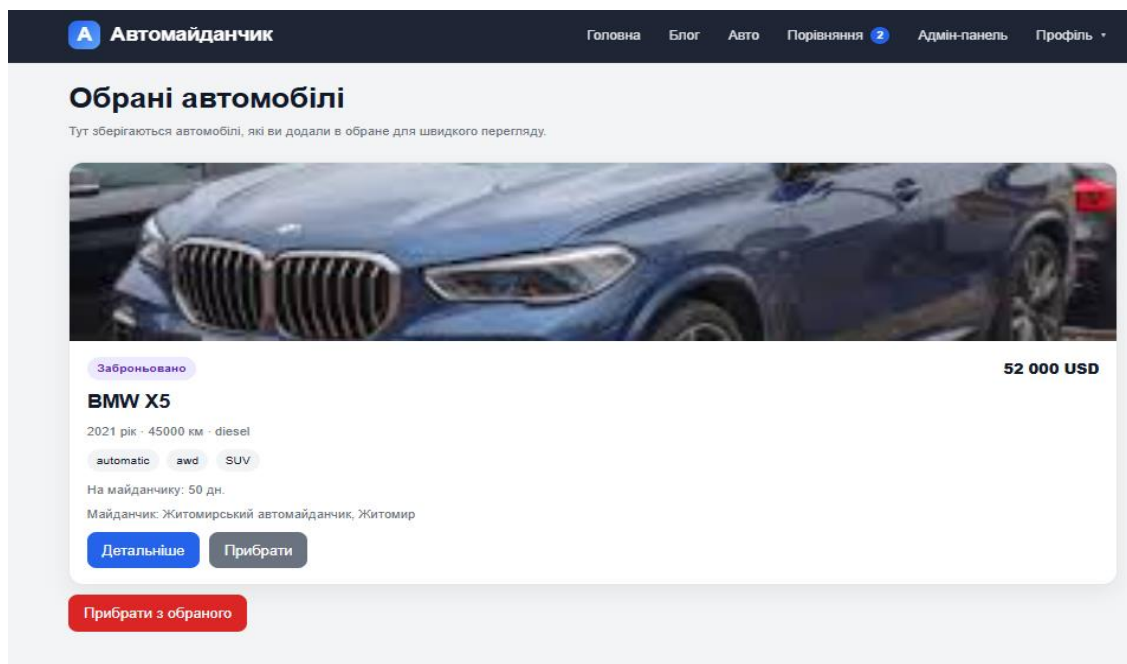


Рисунок 3.6 – Сторінка обраних автомобілів

Функція порівняння автомобілів призначена для прийняття більш обґрунтованого рішення. У таблиці порівняння відображаються основні параметри: ціна, рік випуску, пробіг, тип пального, коробка передач, тип приводу, об'єм двигуна та статус. Це дозволяє користувачу швидко побачити різницю між пропозиціями.

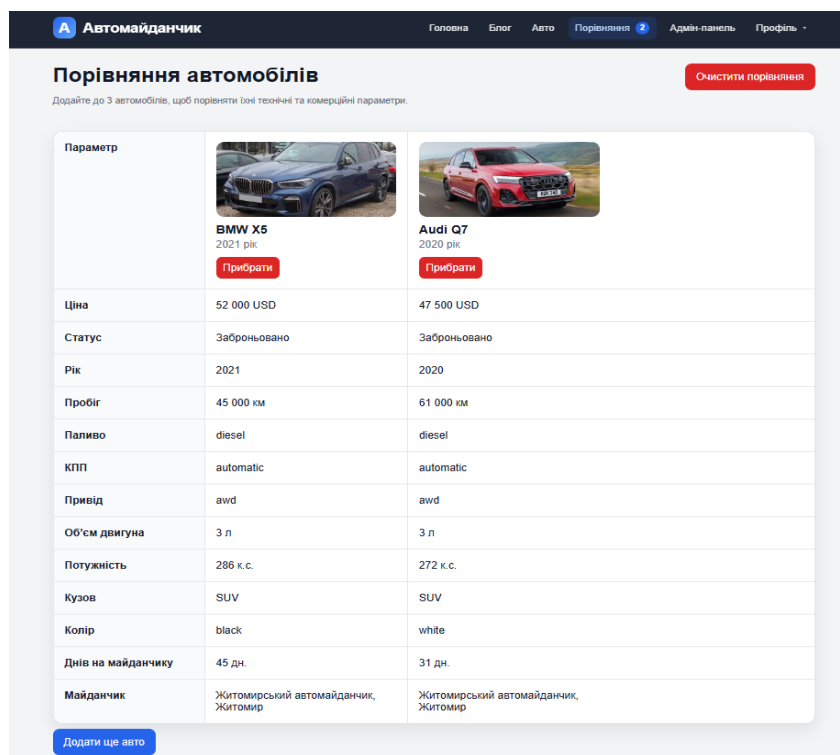


Рисунок 3.7 – Сторінка порівняння автомобілів

Адміністративний інтерфейс платформи

Адміністративний інтерфейс призначений для працівників автомобільного майданчика, які відповідають за наповнення каталогу, актуальність інформації, обробку заявок і оновлення доставки. Доступ до цього інтерфейсу обмежується роллю адміністратора, що забезпечує захист службових функцій від звичайних користувачів.

Головна сторінка адміністративної панелі містить переходи до основних модулів: автомобілів, заявок, доставки, блогу та сповіщень. Такий поділ дозволяє адміністратору швидко переходити до потрібної частини системи та виконувати конкретні операції без зайвих дій.

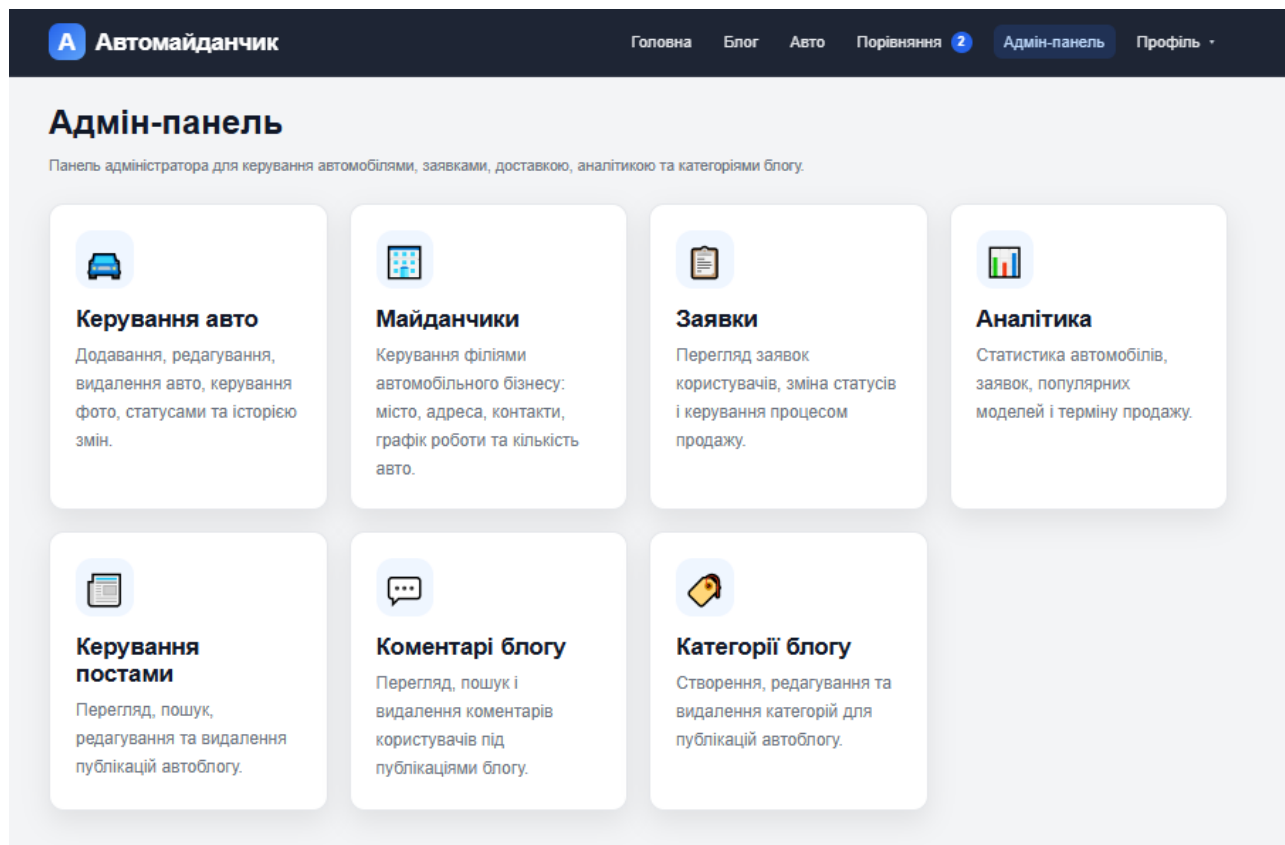


Рисунок 3.8 – Головна сторінка адміністративної панелі

Модуль керування автомобілями відображає таблицю транспортних засобів. У таблиці вказуються основні дані: марка, модель, ціна, статус і дата оновлення. Адміністратор може перейти до редагування, видалити запис або змінити дані про фотографії. Такий підхід спрощує підтримку актуальності каталогу.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		59

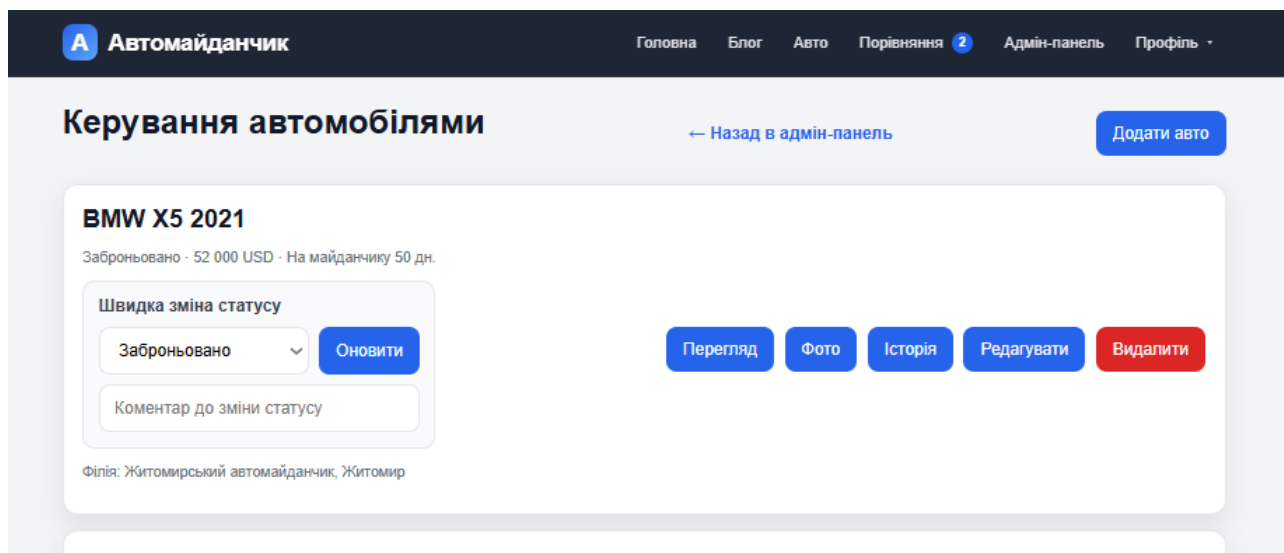


Рисунок 3.9 – Сторінка керування автомобілями

Форма створення автомобіля містить поля для введення технічних і комерційних параметрів. Адміністратор заповнює марку, модель, рік, VIN-код, пробіг, тип пального, коробку передач, привід, ціну, статус, опис і завантажує зображення. Після збереження автомобіль стає доступним у каталозі.

Рисунок 3.10 – Форма створення автомобіля

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		60

Модуль заявок використовується для роботи з клієнтськими зверненнями. Адміністратор бачить, який користувач залишив заявку, на який автомобіль вона створена, який коментар і контактний телефон вказано. Після опрацювання заявки адміністратор змінює її статус і може залишити відповідь.

Автомайданчик Головна Блог Авто Порівняння 2 Адмін-панель Профіль

Заявки на автомобілі

[← Назад в адмін-панель](#)

Усі заявки

Підтверджено

BMW X5 2021

Клієнт: Адміністратор - admin@example.com
Телефон: +380966493072
Створено: 06.06.2026, 20:48:56

Підтверджено

Коментар адміністратора

Оновити статус

Доставка

Переглянути авто

Рисунок 3.11 – Сторінка керування заявками

Модуль доставки використовується для супроводу автомобілів під замовлення. Адміністратор оновлює етапи доставки, додає орієнтовні дати та коментарі. Після оновлення інформації користувач бачить актуальний стан у своєму кабінеті, а система може створити відповідне сповіщення.

Автомайданчик Головна Блог Авто Порівняння 2 Адмін-панель Профіль

Оновити доставку

Після збереження система оновить поточний етап доставки, додасть запис в історію та надішле користувачу повідомлення.

Поточний етап Статус етапу

Доставка В процесі

Орієнтовна дата доставки

дд. мм. гггг

Загальна примітка до доставки

Наприклад: авто передано перевізнику, очікується прибуття на майданчик...

Опис оновлення для історії

Наприклад: автомобіль прибув на митний склад, документи передано брокеру...

Зберегти оновлення доставки

Рисунок 3.12 – Сторінка керування доставкою

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		61

Блог адмініструється через окремий розділ. Адміністратор може створювати інформаційні записи, редагувати заголовок, короткий опис, основний текст і статус публікації. Такий модуль дозволяє підтримувати інформаційну активність платформи та надавати користувачам корисні матеріали.

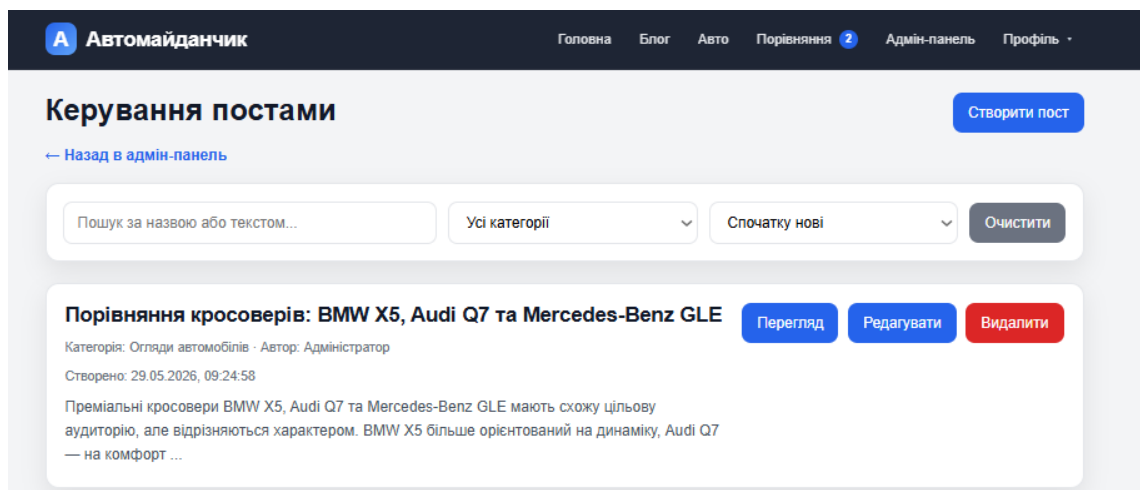


Рисунок 3.13 – Сторінка керування блогом

Таблиця 3.1

Основні сторінки адміністративного інтерфейсу

Сторінка	Призначення	Основні дії
Панель керування	Загальна навігація між модулями	перехід до автомобілів, заявок, доставки та блогу
Автомобілі	Керування каталогом	створення, редагування, видалення, зміна статусу
Заявки	Опрацювання звернень клієнтів	перегляд, зміна статусу, коментар адміністратора
Доставка	Супровід авто під замовлення	оновлення етапів, дат і коментарів
Блог	Керування інформаційними матеріалами	створення, редагування та публікація записів

Порядок оформлення заявки та відстеження доставки

Оформлення заявки починається з детальної сторінки автомобіля. Користувач переглядає характеристики, ціну, статус і фотографії, після чого може натиснути кнопку створення заявки. Система перевіряє, чи користувач

авторизований, і відкриває форму для введення контактної інформації та коментаря.

Після надсилання форми клієнтська частина виконує запит до серверного API. Сервер перевіряє коректність вхідних даних, створює запис заявки в базі даних, прив'язує його до користувача та автомобіля, після чого повертає відповідь про успішне створення. У кабінеті користувача заявка відображається зі статусом, який надалі може змінювати адміністратор.

Для автомобілів під замовлення важливим є відображення доставки. Система використовує послідовність етапів, які відповідають реальному процесу: участь в аукціоні, викуп, відправка, митне оформлення, доставка на майданчик і передача клієнту. Кожен етап має опис, статус виконання, орієнтовну дату завершення та коментар адміністратора.

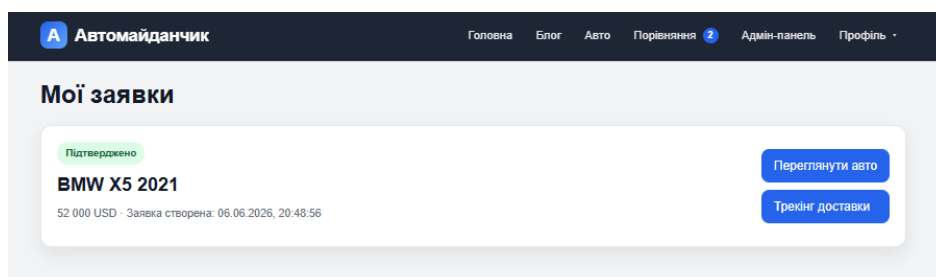


Рисунок 3.14 – Сторінка замовлень користувача

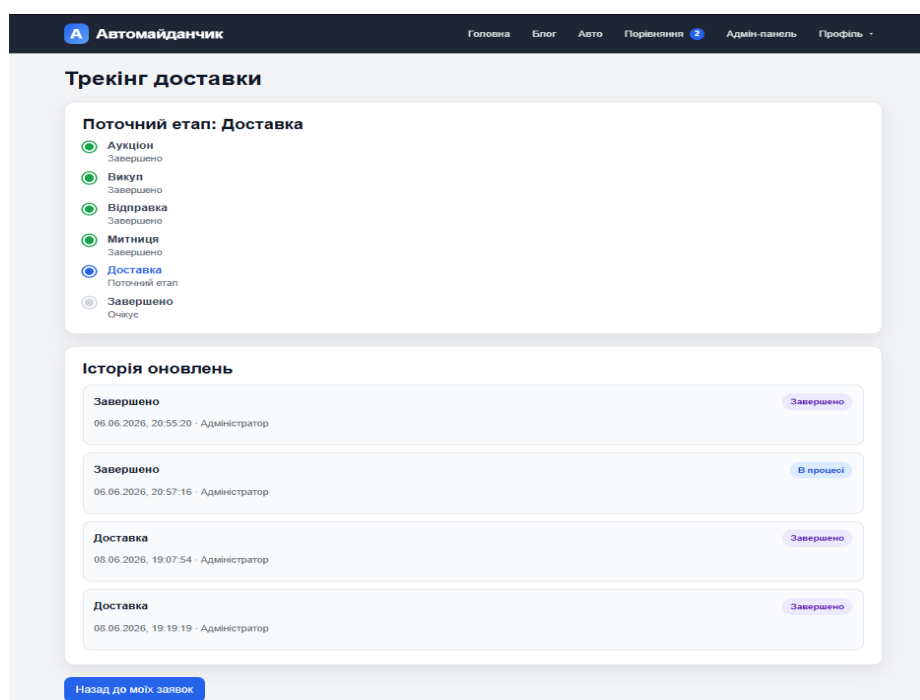


Рисунок 3.15 – Відображення трекінгу доставки автомобіля

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		63

Етапи доставки автомобіля під замовлення

Етап	Опис	Результат
Аукціон	Підбір і участь у торгах за автомобіль	визначено автомобіль для викупу
Викуп	Підтвердження придбання автомобіля	авто закріплено за замовленням
Відправка	Підготовка до транспортування	авто передано перевізнику
Митниця	Проходження митного оформлення	авто допущено до доставки в країні
Доставка	Переміщення на автомобільний майданчик	авто прибуло на майданчик
Видача	Передача автомобіля клієнту	замовлення завершено

Наявність блогу в користувацькій частині доповнює основний функціонал платформи. Через інформаційні статті майданчик може пояснювати умови купівлі, особливості доставки, правила перевірки автомобіля та інші питання, які виникають у потенційних покупців. Це підвищує довіру до платформи та робить її більш корисною для користувача.

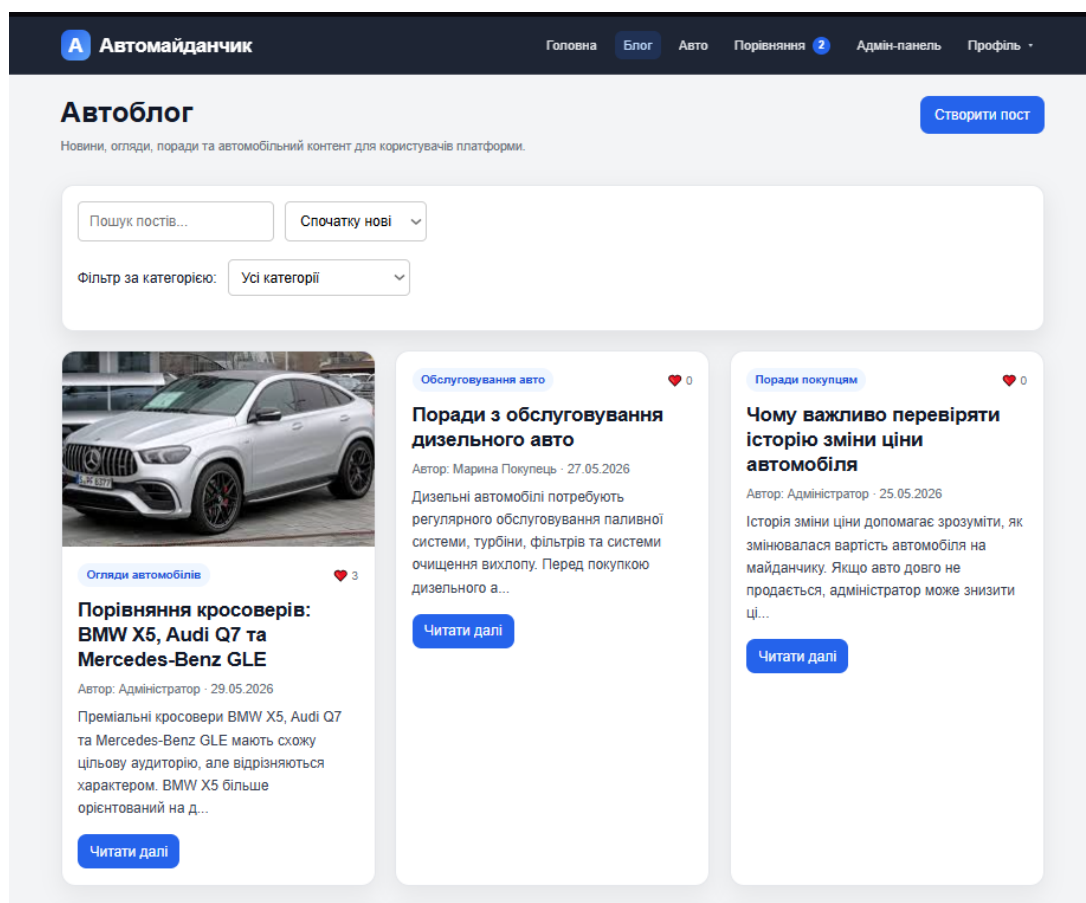


Рисунок 3.16 – Сторінка блогу вебплатформи

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		64

3.3. Тестування роботи програмного комплексу

Тестування вебплатформи проводилося для перевірки працездатності основних функцій системи. Основна увага приділялася не лише авторизації, а всім ключовим сценаріям: перегляду каталогу, фільтрації автомобілів, відкриттю детальної сторінки, створенню заявки, роботі з обраним, порівнянням, оновленню статусів у адміністративній панелі та перевірці доставки.

Під час тестування каталогу перевірялося, чи коректно відображаються автомобілі, чи працюють фільтри за маркою, моделлю, ціною, роком і статусом, а також чи відкривається детальна сторінка обраного автомобіля. Важливою умовою була відповідність даних на клієнтській частині тим даним, які зберігаються в базі.



Рисунок 3.17 – Схема тестування каталогу автомобілів

Під час тестування заявки перевірявся весь шлях користувача: вибір автомобіля, відкриття форми, введення контактних даних, надсилання запиту, створення запису в базі даних, відображення заявки в кабінеті користувача та поява заявки в адміністративній панелі.



Рисунок 3.18 – Схема тестування оформлення заявки

Адміністративна частина тестувалася окремо, оскільки вона містить операції, які впливають на дані платформи. Було перевірено, що звичайний користувач не має доступу до адміністративних маршрутів, а адміністратор може створювати автомобілі, редагувати записи, змінювати статуси заявок і оновлювати доставку.

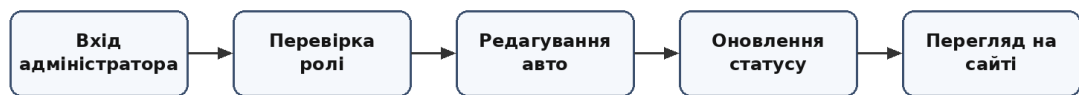


Рисунок 3.19 – Схема тестування адміністративної панелі

На рисунку 3.20 наведено результати аналізу мережевих запитів під час завантаження сторінок вебплатформи за допомогою інструментів розробника браузера. Під час перевірки було встановлено, що клієнтська частина коректно завантажує програмні модулі, стилі, зображення та виконує API-запити до серверної частини. Усі основні запити завершуються успішно зі статусом 200, що підтверджує працездатність вебплатформи.

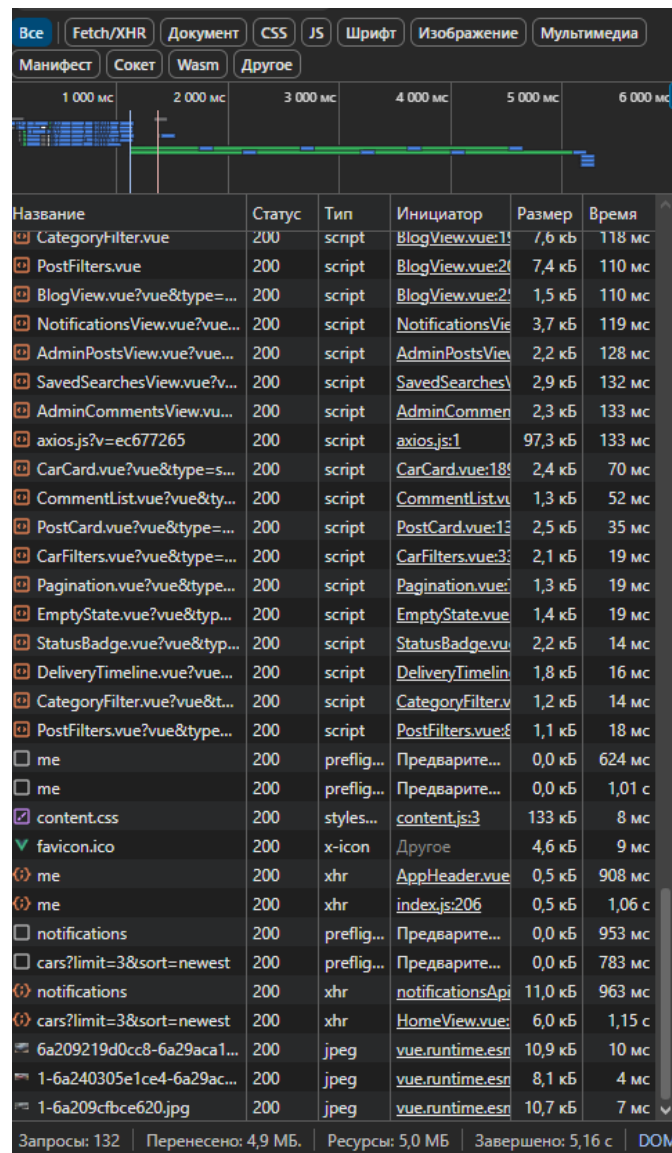


Рисунок 3.20 – Аналіз мережевих запитів під час завантаження сторінок вебплатформи

Таблиця 3.3

Сценарії функціонального тестування

Сценарій	Очікуваний результат	Статус
Перегляд каталогу	Список автомобілів завантажується та відображається з актуальними даними	Успішно
Фільтрація автомобілів	Після вибору параметрів відображаються лише відповідні автомобілі	Успішно
Детальна сторінка	Користувач бачить характеристики, фото, статус і доступні дії	Успішно
Оформлення заявки	Заявка створюється та відображається в кабінеті й адмін-панелі	Успішно
Обране	Автомобіль додається та видаляється зі списку обраного	Успішно
Порівняння	Кілька автомобілів відображаються в таблиці порівняння	Успішно
Оновлення доставки	Користувач бачить новий етап після зміни адміністратором	Успішно
Захист адмін-панелі	Користувач без ролі адміністратора не отримує доступ	Успішно

Таблиця 3.4

Результати тестування модулів

Модуль	Що перевірялося	Результат
Авторизація	Вхід, токен, перевірка ролі	Працює коректно
Каталог	Завантаження списку, фільтри, перехід до деталей	Працює коректно
Заявки	Створення, перегляд, зміна статусу	Працює коректно
Доставка	Оновлення етапів і відображення в кабінеті	Працює коректно
Адмін-панель	Доступ за роллю та керування даними	Працює коректно
Блог	Перегляд і адміністрування записів	Працює коректно

Підсумкова перевірка показала, що основні модулі працюють узгоджено. Каталог отримує дані з API, заявки зберігаються в базі, адміністративна панель дозволяє змінювати статуси, трекінг доставки відображається користувачу, а сповіщення інформують про важливі зміни. Отже, реалізована вебплатформа відповідає поставленим функціональним вимогам.

Окремо перевірялася робота повідомлень про результат дій. Після успішного створення заявки користувач повинен бачити підтвердження, а після

помилки введення - зрозуміле повідомлення. Це важливо для зручності використання, оскільки користувач повинен розуміти, чи була дія виконана, чи потрібно виправити введені дані.

Під час тестування інтерфейсу перевірялася не тільки наявність сторінок, а й зручність переходів між ними. Було важливо переконатися, що користувач може швидко перейти від каталогу до детальної сторінки, від детальної сторінки до заявки, від профілю до власних замовлень, а адміністратор - від панелі керування до автомобілів, заявок і доставки.

Тестування ролей підтверджує правильність розмежування доступу. Гість може переглядати відкриті сторінки, зареєстрований користувач отримує доступ до особистих функцій, а адміністратор може працювати зі службовими модулями. Якщо користувач без відповідної ролі намагається виконати адміністративну дію, система повинна відхилити запит і не змінювати дані.

Під час перевірки API особлива увага приділялася статусам відповідей. Для успішних операцій очікувалися коди 200 або 201, для помилкових запитів - коди 400, 401, 403 або 404 залежно від ситуації. Такий контроль дозволяє переконатися, що серверна частина не тільки виконує правильні дії, а й коректно повідомляє клієнтську частину про результат.

Окремо перевірялися позитивні та негативні сценарії. До позитивних сценаріїв належали успішна авторизація, відкриття каталогу, створення заявки, додавання автомобіля до обраного, оновлення доставки та створення публікації блогу. До негативних сценаріїв належали спроба створити заявку без авторизації, введення некоректного телефону, доступ звичайного користувача до адміністративної панелі та відкриття неіснуючого запису.

Тестування виконувалося за сценарним підходом. Для кожного основного модуля було визначено послідовність дій, очікуваний результат і фактичну реакцію системи. Такий підхід дозволяє перевірити не тільки окремі кнопки або форми, а повний шлях користувача від відкриття сторінки до зміни стану даних у системі.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 - ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		68

Висновки до третього розділу

У третьому розділі було описано інтерфейс і порядок роботи з вебплатформою для автомобільних майданчиків. Розглянуто користувацьку частину, яка включає головну сторінку, каталог, детальну сторінку автомобіля, авторизацію, особистий кабінет, обране, порівняння, заявки, доставку та блог.

Також було описано адміністративний інтерфейс, який забезпечує керування автомобілями, заявками, доставкою та записами блогу. Показано, що адміністративна панель є основним інструментом підтримки актуальності даних на платформі та супроводу взаємодії з клієнтами.

У межах тестування було перевірено основні сценарії роботи системи. Перевірка охопила каталог, фільтрацію, заявки, обране, порівняння, доставку, адміністративну панель і захист маршрутів за ролями. Результати тестування підтвердили працездатність реалізованого функціоналу.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		69

ВИСНОВКИ

У результаті виконання випускної кваліфікаційної роботи бакалавра було розроблено вебплатформу для автомобільних майданчиків, яка поєднує публічний каталог автомобілів, особистий кабінет користувача та адміністративну панель. Розроблений програмний продукт спрямований на автоматизацію процесів обліку, продажу, оформлення заявок і супроводу доставки автомобілів.

У роботі було виконано аналіз предметної області та визначено основні проблеми, які виникають під час роботи автомобільних майданчиків у цифровому середовищі. Було встановлено, що для ефективної роботи потрібна не лише сторінка з оголошеннями, а повноцінна інформаційна система з обліком статусів, історією змін, заявками, доставкою та персоналізованими функціями користувача.

Було проаналізовано наявні аналоги програмного продукту та визначено, що більшість із них орієнтована на пошук і публікацію оголошень. На основі цього сформовано підхід до розробки власної системи, яка поєднує переваги автомобільного каталогу та адміністративного інструмента для керування майданчиком.

У процесі проєктування було обґрунтовано використання клієнт-серверної архітектури, REST API, реляційної бази даних і рольового розмежування доступу. Для реалізації серверної частини використано Symfony, для клієнтської частини Vue, для зберігання даних MySQL, а для роботи із сутностями Doctrine ORM.

У межах реалізації було створено основні модулі системи: каталог автомобілів, детальну сторінку авто, авторизацію, особистий кабінет, заявки, обране, порівняння, доставку, сповіщення, збережені пошуки, блог і адміністративну панель. Реалізований функціонал дозволяє користувачу зручно працювати з пропозиціями, а адміністратору підтримувати актуальність даних.

Проведене тестування підтвердило працездатність основних сценаріїв вебплатформи. Було перевірено каталог, фільтрацію, створення заявок,

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		70

оновлення статусів, роботу адміністративної панелі, відображення доставки та обмеження доступу до службових функцій. Отримані результати показують, що розроблена система може бути використана як основа для подальшого впровадження в діяльність автомобільного майданчика або автодилера.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		71

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. AUTO.RIA: онлайн-сервіс пошуку та продажу автомобілів. URL: <https://auto.ria.com/uk/> (дата звернення: 10.06.2026).
2. OLX Авто. : автомобільний розділ. URL: <https://www.olx.ua/uk/transport/legkovye-avtomobili/> (дата звернення: 10.06.2026).
3. Copart. Online Vehicle Auctions. URL: <https://www.copart.com/> (дата звернення: 10.06.2026).
4. Pressman R. S., Maxim B. R. Software Engineering: A Practitioner's Approach. 9th ed. New York : McGraw-Hill Education, 2020. 704 p.
5. Symfony Documentation. Symfony : офіційна документація. URL: <https://symfony.com/doc/current/index.html> (дата звернення: 10.06.2026).
6. Doctrine ORM Documentation. Doctrine Project : офіційна документація. URL: <https://www.doctrine-project.org/projects/orm.html> (дата звернення: 10.06.2026).
7. LexikJWTAuthenticationBundle Documentation. Symfony Bundle Documentation. URL: <https://symfony.com/bundles/LexikJWTAuthenticationBundle/current/index.html> (дата звернення: 10.06.2026).
8. Vue.js Documentation. Vue.js : офіційна документація. URL: <https://vuejs.org/guide/introduction.html> (дата звернення: 10.06.2026).
9. Vue Router Documentation. Vue.js : офіційна документація. URL: <https://router.vuejs.org/> (дата звернення: 10.06.2026).
10. Vite Documentation. Vite : офіційна документація. URL: <https://vite.dev/guide/> (дата звернення: 10.06.2026).
11. Axios Documentation. Axios : офіційна документація. URL: <https://axios-http.com/docs/intro> (дата звернення: 10.06.2026).

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		72

12. MySQL 8.4 Reference Manual. Oracle : офіційна документація. URL: <https://dev.mysql.com/doc/refman/8.4/en/> (дата звернення: 10.06.2026).

13. PHP Manual. PHP Group : офіційна документація. URL: <https://www.php.net/manual/en/> (дата звернення: 10.06.2026).

14. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT). RFC 7519. 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата звернення: 10.06.2026).

15. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures : doctoral dissertation. University of California, Irvine, 2000. URL: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm> (дата звернення: 10.06.2026).

16. Fielding R., Nottingham M., Reschke J. HTTP Semantics. RFC 9110. 2022. URL: <https://www.rfc-editor.org/rfc/rfc9110> (дата звернення: 10.06.2026).

17. OWASP Foundation. OWASP Top 10:2021. URL: <https://owasp.org/Top10/> (дата звернення: 10.06.2026).

18. OWASP Foundation. Application Security Verification Standard. URL: <https://owasp.org/www-project-application-security-verification-standard/> (дата звернення: 10.06.2026).

19. OWASP Foundation. Cheat Sheet Series. URL: <https://cheatsheetseries.owasp.org/> (дата звернення: 10.06.2026).

20. W3C. Web Content Accessibility Guidelines (WCAG) 2.2. URL: <https://www.w3.org/TR/WCAG22/> (дата звернення: 10.06.2026).

21. Mozilla Developer Network. HTTP. MDN Web Docs. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP> (дата звернення: 10.06.2026).

22. Mozilla Developer Network. JavaScript Guide. MDN Web Docs. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide> (дата звернення: 10.06.2026).

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		73

23. Mozilla Developer Network. HTML: HyperText Markup Language. MDN Web Docs. URL: <https://developer.mozilla.org/en-US/docs/Web/HTML> (дата звернення: 10.06.2026).

24. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Boston : Addison-Wesley, 1994. 395 p.

25. Martin R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River : Prentice Hall, 2008. 464 p.

26. Fowler M. Patterns of Enterprise Application Architecture. Boston : Addison-Wesley, 2002. 560 p.

27. Kruchten P. The 4+1 View Model of Architecture. IEEE Software. 1995. Vol. 12, No. 6. P. 42-50. DOI: 10.1109/52.469759.

28. ISO/IEC/IEEE 29148:2018. Systems and software engineering - Life cycle processes - Requirements engineering. Geneva : ISO, 2018.

29. ISO/IEC 25010:2011. Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models. Geneva : ISO, 2011.

30. ДСТУ 8302:2015. Бібліографічне посилання. Загальні положення та правила складання. Київ : ДП «УкрНДНЦ», 2016. 16 с.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		74

ДОДАТКИ

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		75

Структура проекту містить дві основні частини: серверний застосунок autoblog і клієнтський застосунок autoblog-front. Серверна частина включає каталоги config, migrations, public, src, templates і tests. Клієнтська частина включає каталоги src/api, src/components, src/composables, src/router і src/views.

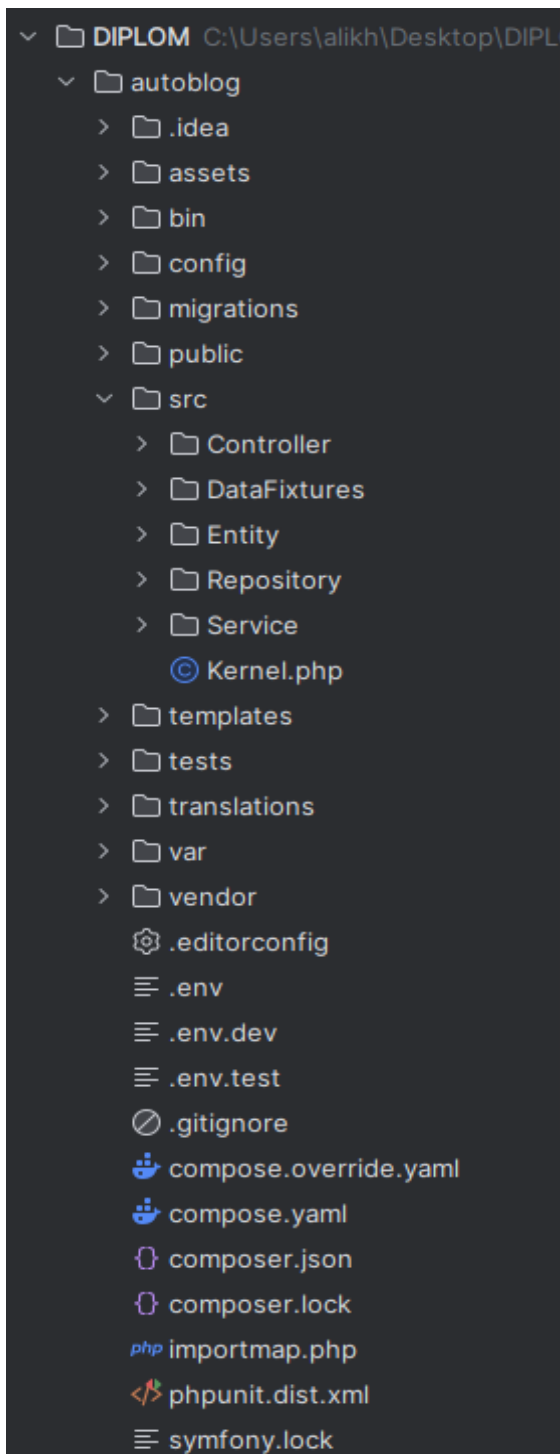


Рисунок А.1 – Структура серверного застосунку

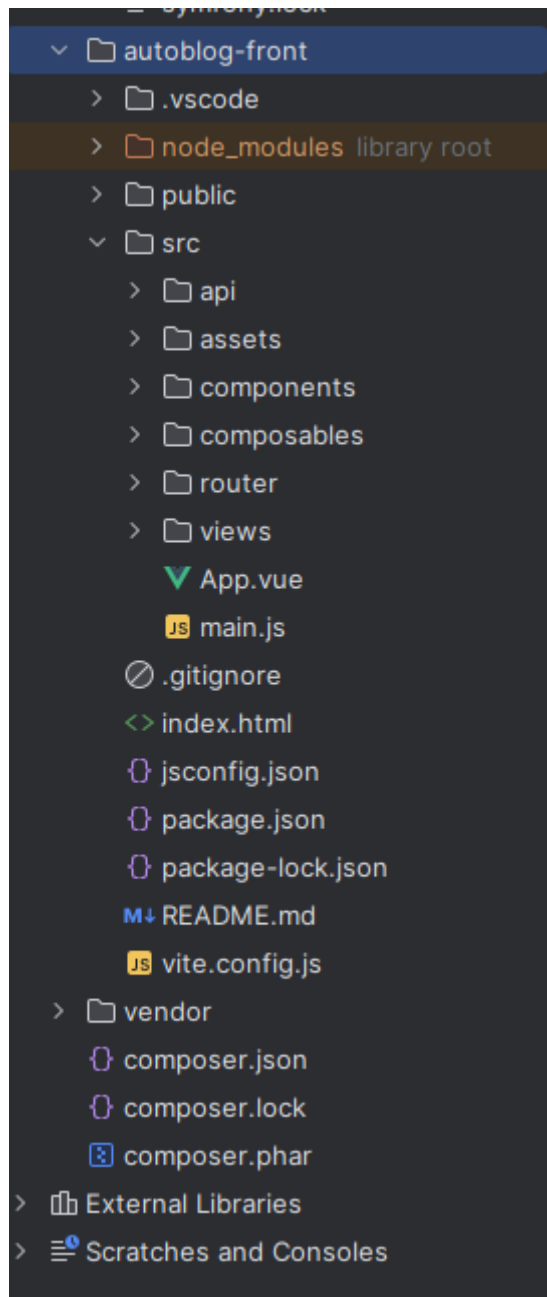


Рисунок А.2 – Структура клієнтського застосунку

Додаток Б

Основні API-маршрути та результати контрольного тестування

У додатку наведено скорочений перелік основних маршрутів серверної частини та контрольні перевірки, які підтверджують працездатність ключових функцій вебплатформи. Повний програмний код зберігається у репозиторії проекту, тому в пояснювальну записку винесено лише узагальнені технічні матеріали, необхідні для перевірки реалізації.

		Бекіров А.Е.			ДУ «Житомирська політехніка».26.121.03.000 – ПЗ	Арк.
		Власенко О.В.				77
Змн.	Арк.	№ докум.	Підпис	Дата		

Основні API-маршрути вебплатформи

Маршрут	Метод	Призначення
/api/register	POST	реєстрація користувача
/api/login	POST	автентифікація та отримання JWT
/api/cars	GET, POST	отримання каталогу та створення автомобіля адміністратором
/api/cars/{id}	GET, PUT, DELETE	перегляд, редагування та видалення автомобіля
/api/cars/{id}/orders	POST	створення заявки на автомобіль
/api/my-orders	GET	перегляд заявок користувача
/api/admin/orders	GET	перегляд заявок в адміністративній панелі
/api/orders/{id}/tracking	GET	перегляд статусу доставки
/api/admin/orders/{id}/tracking	POST	створення або оновлення трекінгу доставки
/api/notifications	GET, PATCH	перегляд і позначення сповіщень
/api/posts	GET, POST	робота з публікаціями блогу
/api/admin/analytics	GET	отримання аналітичних даних адміністратора

Таблиця Б.2

Результати контрольного тестування вебплатформи

Сценарій	Очікуваний результат	Статус
Перегляд каталогу автомобілів	завантажується список автомобілів із фото, ціною та статусом	успішно
Фільтрація каталогу	відображаються автомобілі, що відповідають заданим параметрам	успішно
Створення заявки	створюється запис заявки та відображається повідомлення користувачу	успішно
Оновлення статусу заявки	адміністратор змінює статус, користувач отримує сповіщення	успішно
Трекінг доставки	відображається поточний етап доставки та історія змін	успішно
Перевірка доступу	звичайний користувач не має доступу до адміністративних маршрутів	успішно